Practical - 1

Practical Title: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition: Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

A. Theory

1. What is the URL and parts of the URL?

Answer: A URL (Uniform Resource Locator**)** is the address of a web page on the internet.

Parts**:** Protocol (https://), Domain Name (example.com), Path (/page.html).

2. How does an HTML file process in a web browser?

Answer: The browser reads the HTML file, creates the page structure (DOM), applies CSS for styling, executes JavaScript if present, and then displays the web page to the user.

3. How will page navigation flow be managed among all HTML pages?

Answer: Page navigation is managed using HTML hyperlinks **(**`<a>` tags). Users can move between pages like Home, Login, Register, Dashboard, and other sections through these links.

4. How will GitHub commits be maintained after each practical?

Answer: After completing each practical, the changes will be committed with a clear and meaningful commit message and then pushed to the GitHub repository to maintain version history and track progress.

B. Implementation

- Identified the StudentHub Portal problem scope and user roles.
- Planned the key modules and navigation flow.
- Created a sitemap with 10+ pages.
- Designed low-fidelity wireframes.
- Organized the project folder structure.
- Created a README file and initialized a GitHub repository.
- Learned about different commands of git

Commands for git:

git status

Git add .

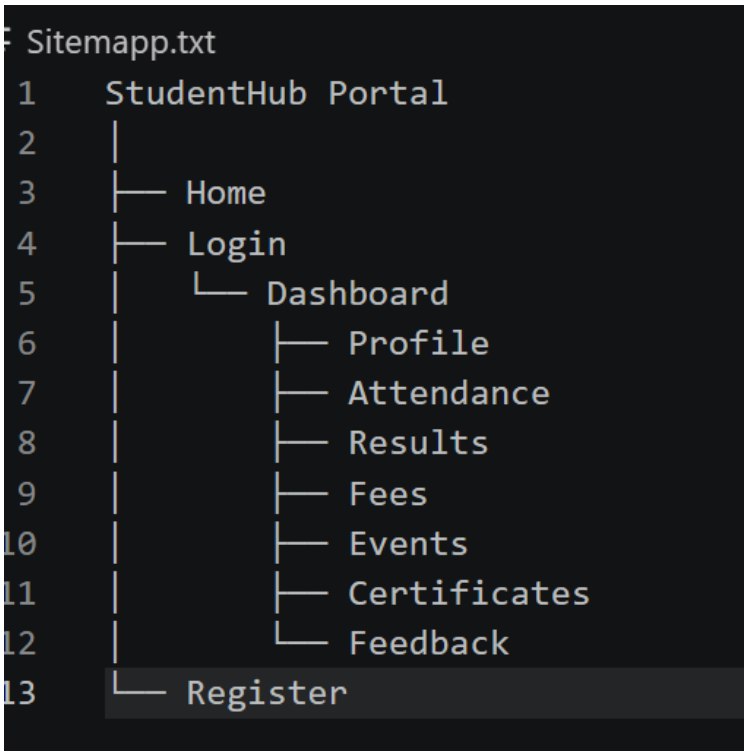
git commit

git pull origin main

git push origin main

C. Evidence of Work (Screenshots)

Figure 1.1 Sitemap



Description: This sitemap shows the navigation flow of studenthub portal.

Figure 1.2 Wireframes

Wireframe - 11



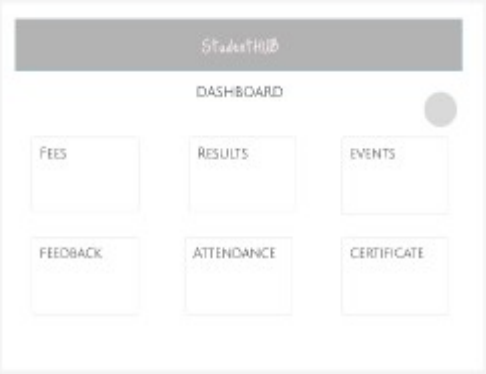
Wireframe - 3

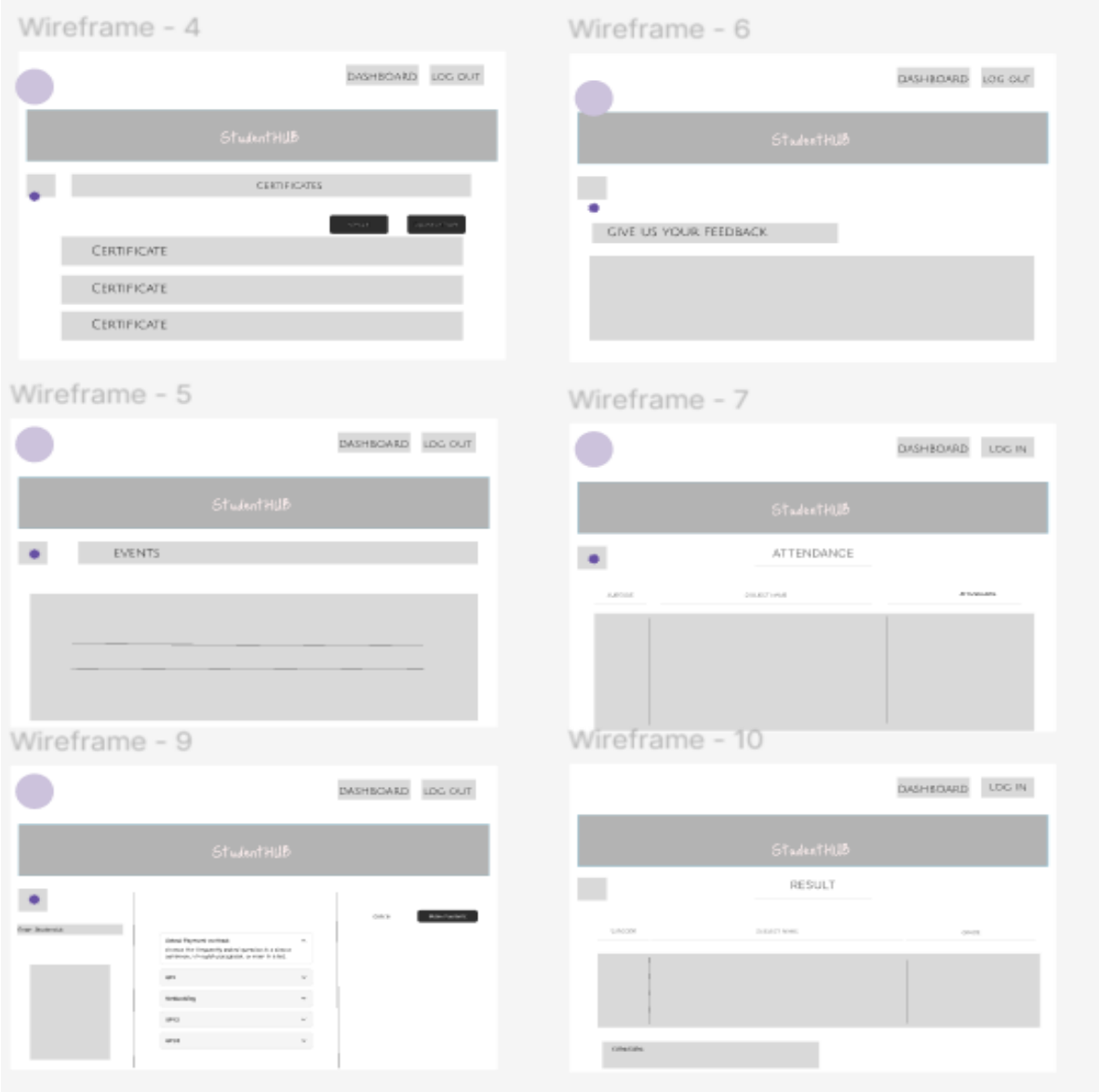


Wireframe - 1



Wireframe - 8





Description: This is wireframes for the studentHub portal which shows how each page will look like .

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

Initially found it difficult to decide the overall flow of the StudentHub Portal and how different pages should be connected .

F. Learning Outcomes:

By analyzing the project requirements and refining the structure step by step, I was able to create a clear navigation flow and well-organized project layout.
And also learned all the commands of github and git for committing changes, connecting repo to vs code editor .

eg. git add ., git commit -m “”, git push, git status

Faculty Remarks:

Practical - 2

Practical Title: Semantic HTML5 Pages with Accessibility-Ready Structure

Problem Definition: Develop static HTML5 skeletons for at least 10 StudentHub pages such as Home, About, Register, Login, Dashboard, Events, Profile, Contact, Admin, FAQ, and Feedback. Use semantic tags and accessibility-friendly structure.

A. Theory

1. Are semantic tags such as header, nav, main, section, article, aside, and footer used properly?

Answer: Yes, Semantic HTML tags are used to organize the webpage into meaningful sections, making the code more structured and easier to understand.

2.Are form labels, headings, image alt text, and navigation links accessible?
Answer: Yes. Proper labels, headings, image `alt` attributes, and clear navigation links are used to improve accessibility and enhance the user experience.

3. Is the page structure consistent across all pages?
Answer: Yes. All pages follow a consistent layout with a similar header, navigation, content section, and footer, ensuring a uniform design throughout the website.

B. Implementation

Made StudentHub portal html pages using these tags :
header – Website header

nav – Navigation menu

aside – Sidebar

main – Main page content

section – Groups related content

div – Layout container

a – Navigation links

form – Login/Register forms

input – User input fields

button – Buttons

table, tr, th, td – Attendance and Results tables

img – Profile images and logos

footer – Footer section

Basic html structure:

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Profile</title>
</head>
<body>
   <h1>Student Profile</h1>
   <p>Name: Priyanshu Kacha</p>
   <p>Course: Computer Engineering</p>
</body>
</html>
```

Form using html:

```
<form>
   <label for="name">Name:</label>
   <input type="text" id="name" placeholder="Enter your name">
   <br><br>
   <label for="email">Email:</label>
   <input type="email" id="email" placeholder="Enter email">
   <br><br>
   <button type="submit">Submit</button>
```

```
</form>
```

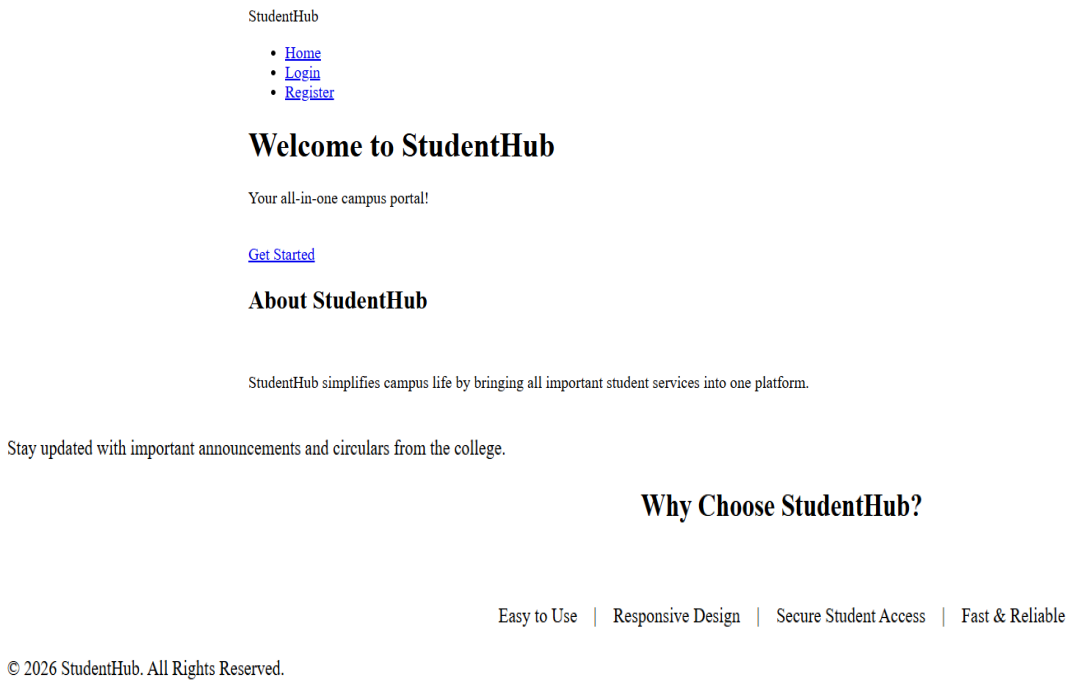
Navigation bar:

```
<nav>
   <a href="index.html">Home</a>
   <a href="dashboard.html">Dashboard</a>
   <a href="register.html">Register</a>
</nav>
```

C. Evidence of Work (Screenshots)

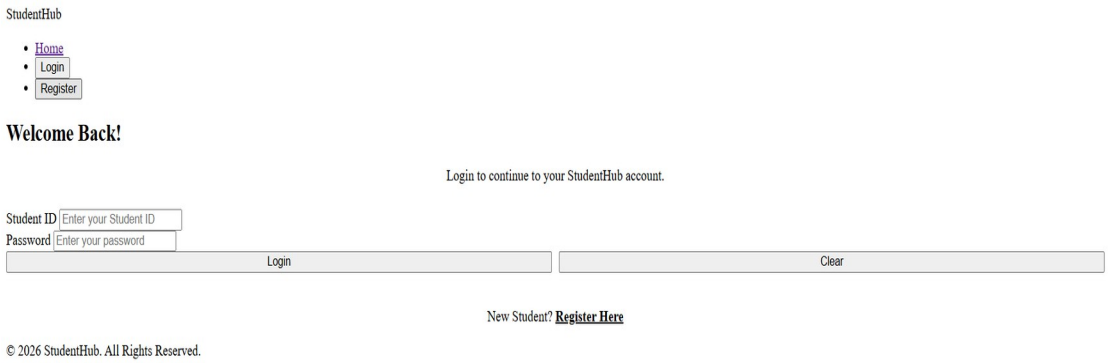
These screenshots of html are taken after modifying code for css. For now , the link to css was not applied .

Figure 1.1 Index.html



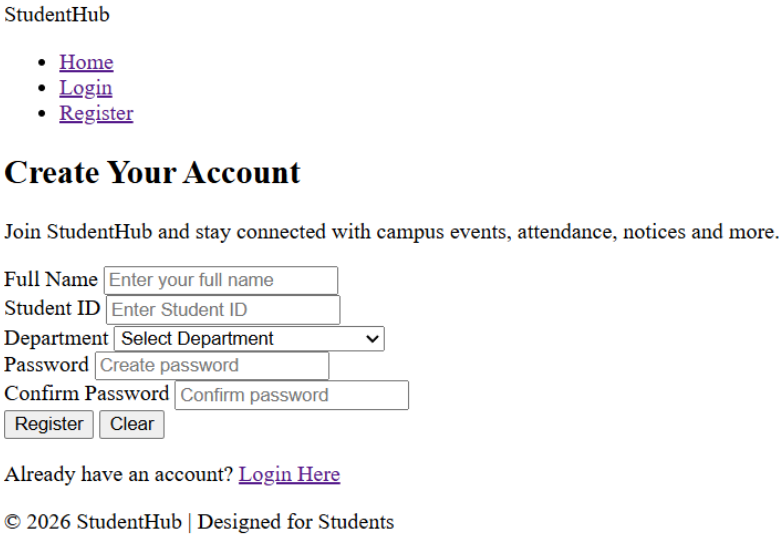
Description: Home page of website

Figure 1.2 login.html



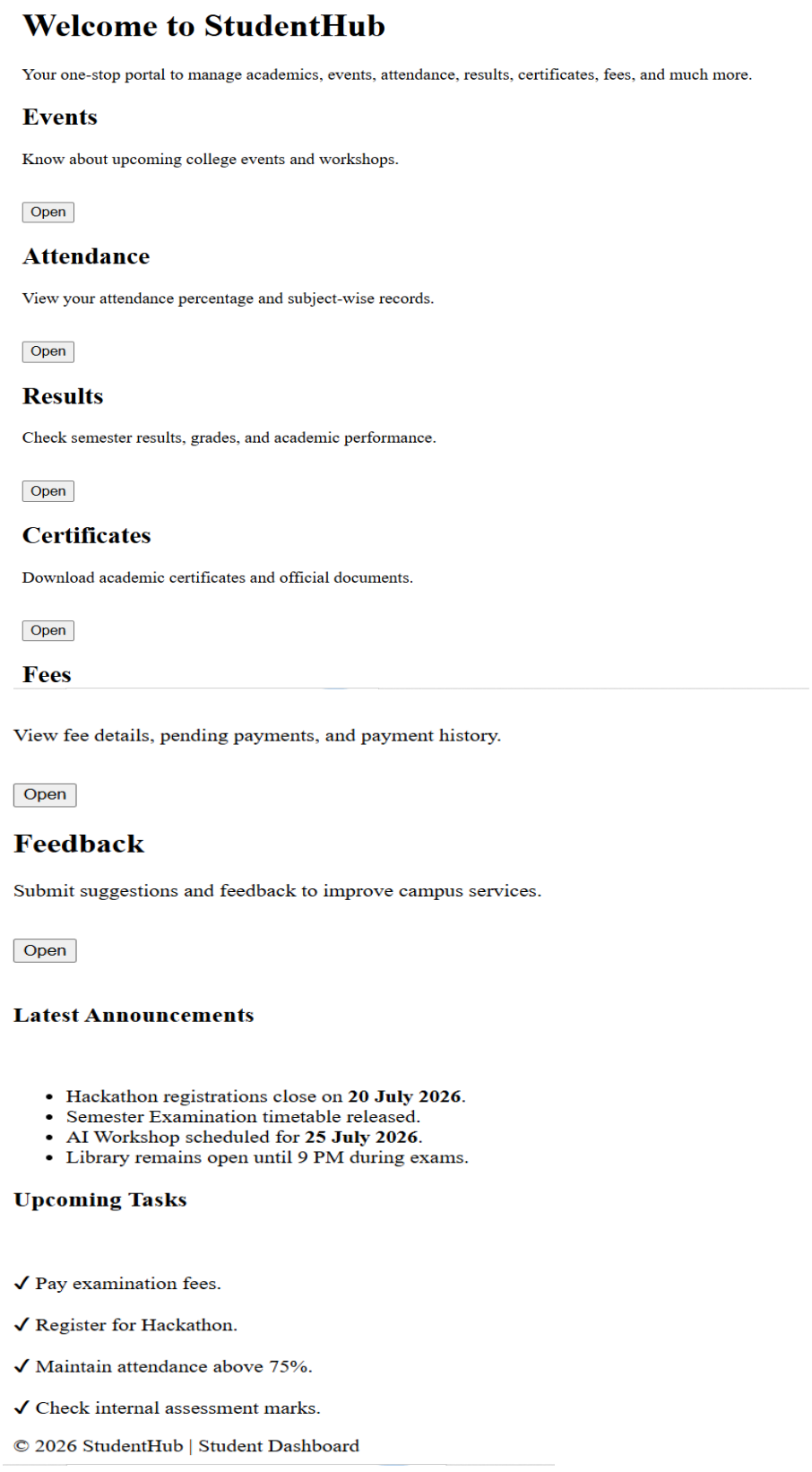
Description: login page for registered users

Figure 1.3 register.html



Description: Register page for new users

Figure 1.4 Dashboard.html



Welcome to StudentHub

Your one-stop portal to manage academics, events, attendance, results, certificates, fees, and much more.

Events

Know about upcoming college events and workshops.

Open

Attendance

View your attendance percentage and subject-wise records.

Open

Results

Check semester results, grades, and academic performance.

Open

Certificates

Download academic certificates and official documents.

Open

Fees

View fee details, pending payments, and payment history.

Open

Feedback

Submit suggestions and feedback to improve campus services.

Open

Latest Announcements

- Hackathon registrations close on **20 July 2026**.
- Semester Examination timetable released.
- AI Workshop scheduled for **25 July 2026**.
- Library remains open until 9 PM during exams.

Upcoming Tasks

✓ Pay examination fees.

✓ Register for Hackathon.

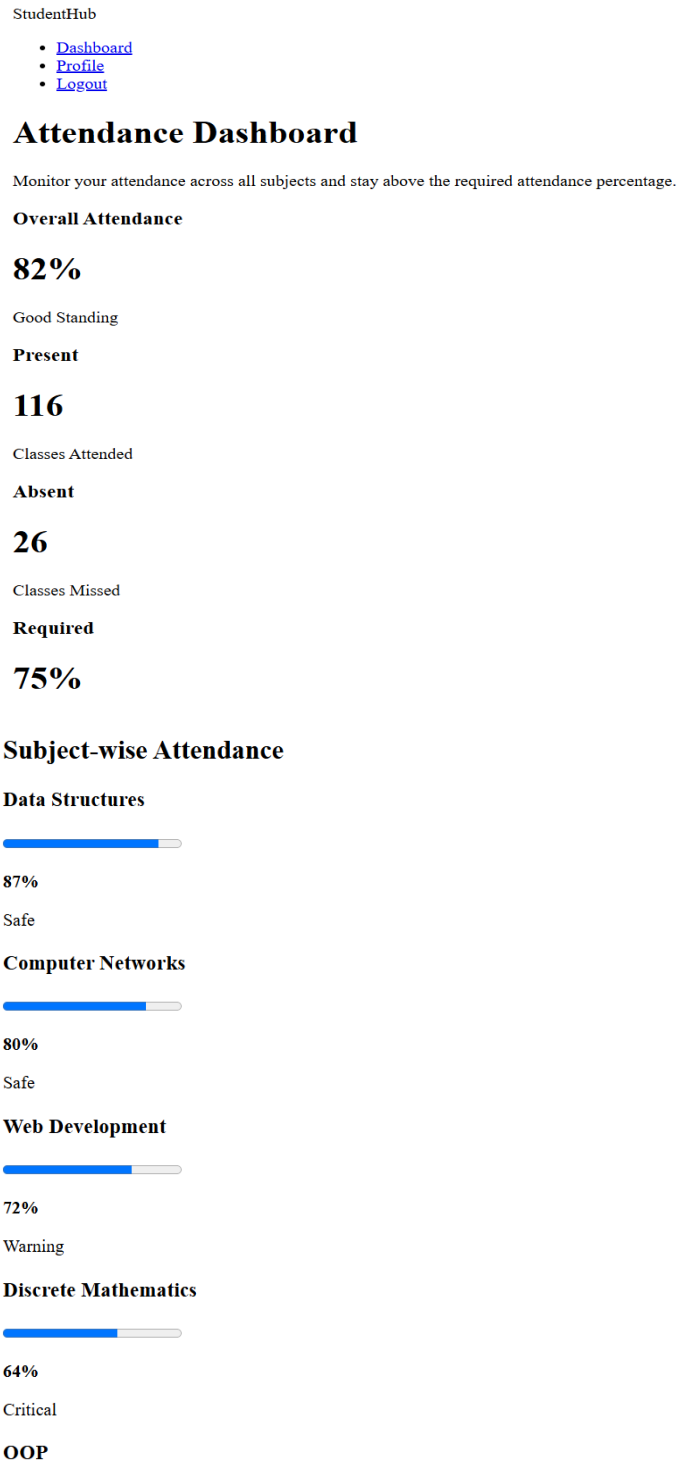
✓ Maintain attendance above 75%.

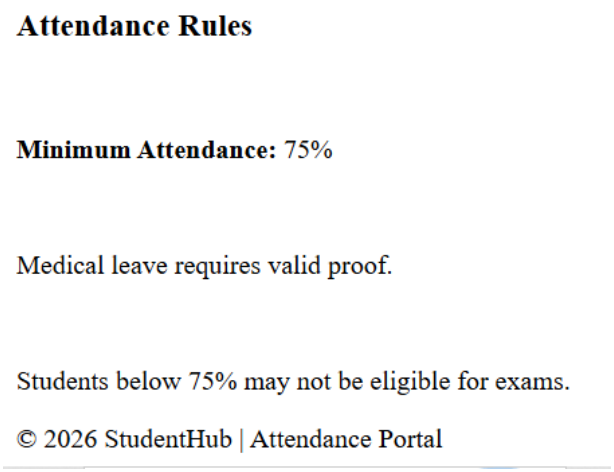
✓ Check internal assessment marks.

© 2026 StudentHub | Student Dashboard

Description: Html page for dashboard when user successfully logs in

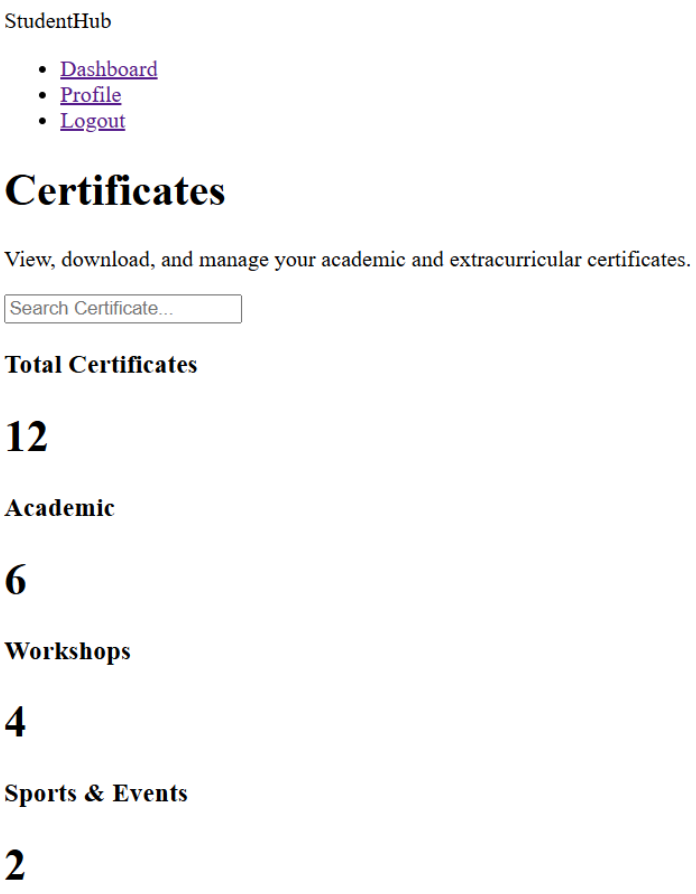
Figure 1.5 Attendance.html





Description: This attendance page include reports of student's attendance including progress bar

Figure 1.6 Certificate.html



Available Certificates

Certificate	Category	Date	Status	Action
Semester 1 Completion	Academic	20 Jan 2026	Available	Download
Web Development Workshop	Workshop	12 Feb 2026	Available	Download
Hackathon Participation	Competition	5 Mar 2026	Processing	Pending
Sports Meet	Sports	18 Apr 2026	Available	Download
NSS Volunteer	Community	8 May 2026	Not Issued	Request

Request Certificate

Need a certificate that isn't available? Submit a request to the administration.

Request Now

Verify Certificate

Verify the authenticity of your certificates using StudentHub verification services.

Verify

Download All

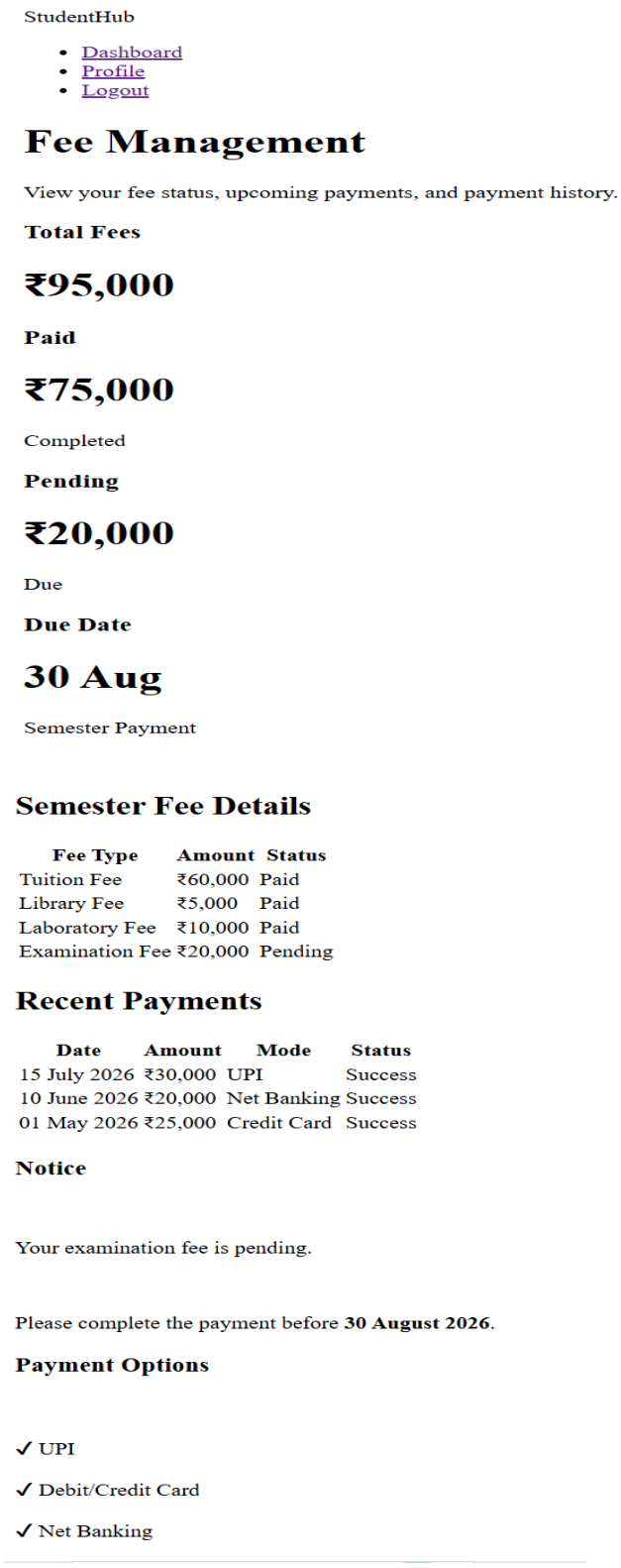
Download all available certificates together as a ZIP archive.

Download All

© 2026 StudentHub. All Rights Reserved.

Description: This html page includes certificate download system for events that student had participated in

Figure 1.7 Fees.html



StudentHub

- Dashboard
- Profile
- Logout

Fee Management

View your fee status, upcoming payments, and payment history.

Total Fees

₹95,000

Paid

₹75,000

Completed

Pending

₹20,000

Due

Due Date

30 Aug

Semester Payment

Semester Fee Details

Fee Type	Amount	Status
Tuition Fee	₹60,000	Paid
Library Fee	₹5,000	Paid
Laboratory Fee	₹10,000	Paid
Examination Fee	₹20,000	Pending

Recent Payments

Date	Amount	Mode	Status
15 July 2026	₹30,000	UPI	Success
10 June 2026	₹20,000	Net Banking	Success
01 May 2026	₹25,000	Credit Card	Success

Notice

Your examination fee is pending.

Please complete the payment before **30 August 2026**.

Payment Options

✓ UPI

✓ Debit/Credit Card

✓ Net Banking

Description: fees.html contains fees details and recent payments of user

Figure 1.8 Events.html

StudentHub

- Dashboard
- Profile
- Logout

College Events

Stay updated with workshops, competitions, festivals, and campus activities.

Search Events

Search Events...

Upcoming Deadline

Hackathon Registration closes on **20 July 2026**.

Register Soon

AI Workshop

Date: 25 July 2026

Venue: Seminar Hall

Faculty: Prof. Mehta

Contact: 9876543210

Upcoming

Upcoming

Sports Fest

Date: 15–18 July

Venue: Sports Ground

Contact: Sports Committee

Phone: 9123456780

Ongoing

Cultural Fest

Date: 10 July

Venue: Auditorium

Contact: Cultural Committee

Phone: 9988776655

Completed

Latest News

- Robotics Competition registrations are now open.
- Independence Day celebrations begin next week.
- Photography Club welcomes new members.
- Hackathon prize pool has been increased.

Event Calendar - July 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
	1	2	3	4	5	6
7	8	9	**10** Cultural Fest	11	12	13
14	**15** Sports Fest	16	17	18	19	**20** Deadline
21	22	23	24	**25** AI Workshop	26	27

Event Coordinators

Student Activity Cell

Email: events@studenthub.edu

Phone: +91 98765 43210

Quick Tips

✓ Register before deadlines.

✓ Carry your Student ID.

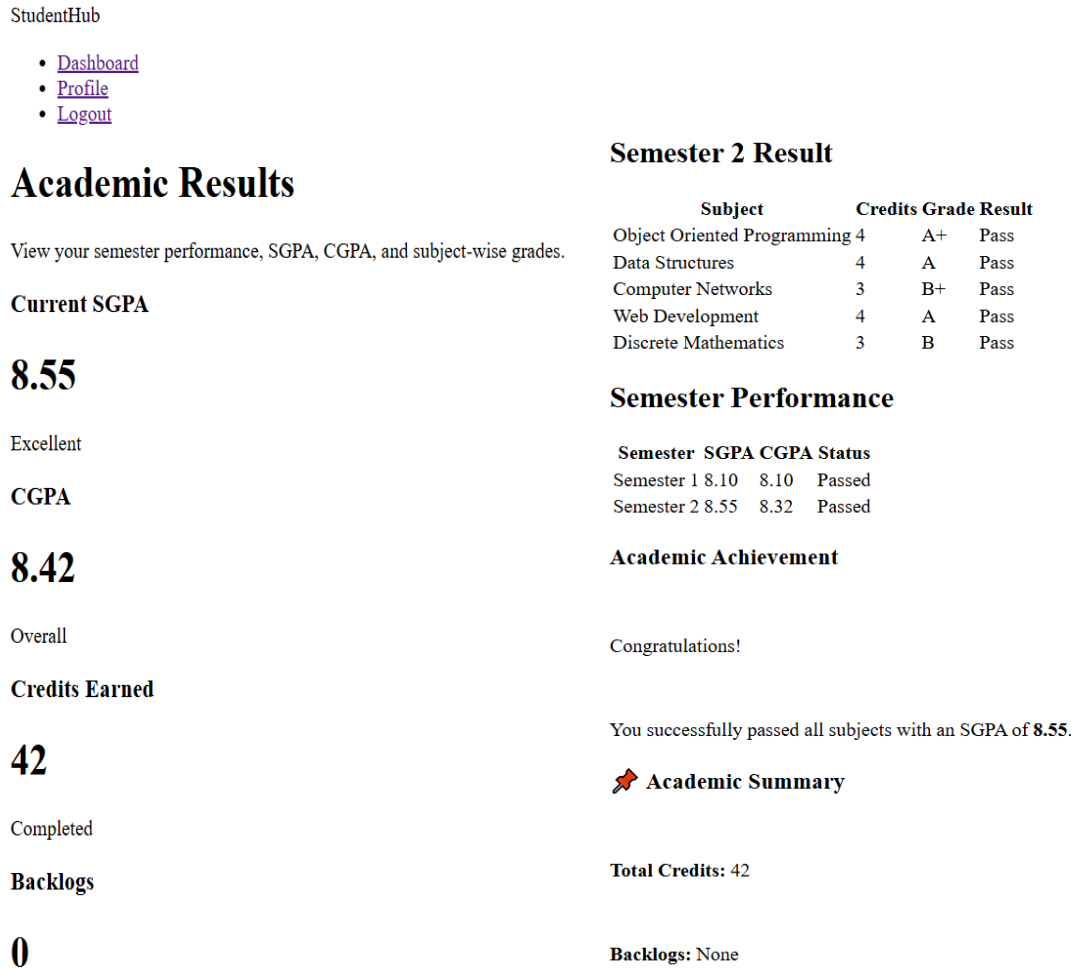
✓ Reach the venue 15 minutes early.

Register Now

© 2026 StudentHub | Events Portal

Description: Events page contains latest updates about events

Figure 1.9 result.html



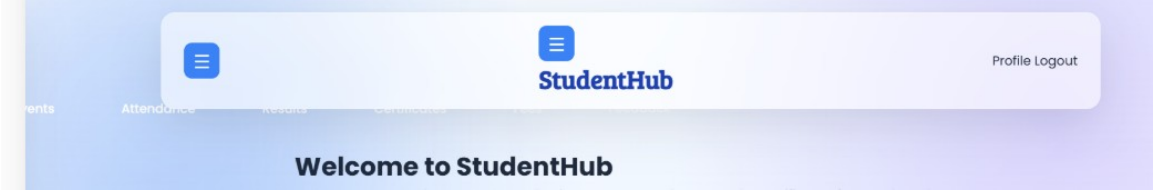
Description: This page contains student's marksheet and results updates

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

I faced problems like proper alignment of tables and different headings setting
I also faced problems while making a sidebar using an aside tag, as it kept being on other pages while hiding them.



(sidebar was added after applying css .

Problem: It kept coming in horizontal style instead of vertical . **)**

F. Learning Outcomes:

I learned to use different html tags and made html pages using that for studenthub portal Then I explored some solutions for it and found an overlay method.

Also learned about progress tag . It is used for showing progress in systems like we have for our phone’s brightness system.

I also learned how a sidebar is made using a aside tag, if you want to make one for all, js and live server will be needed but since i did not use live server for that, i added it in each page’s html.

Faculty Remarks:

Practical - 3

Practical Title: Responsive UI Design using CSS Grid, Flexbox, and Bootstrap/Tailwind

Problem Definition: Design responsive layouts for Home, About, Registration, Dashboard, and Events pages using CSS Grid and Flexbox. Optional to use CSS framework such as Bootstrap or Tailwind. Ensure mobile-first responsiveness.

A. Theory

1. How does the layout adapt for mobile, tablet, and desktop screens?
Answer: The StudentHub layout uses responsive design. The content, cards, navigation, and sidebar adjust their size and position according to the screen size. It works properly on mobile, tablet, and desktop screens.

2. Which layout technique is used and why?
Answer: CSS Flexbox and Grid are mainly used. Flexbox helps align elements such as the navigation and buttons, while Grid is useful for arranging dashboard cards. They make the layout flexible and responsive.

3. Are typography, spacing, colors, and contrast readable?
Answer: Yes. Poppins and Bree Serif are used for clear typography. Proper spacing is maintained between sections and cards. The selected colors provide good contrast and make the text easy to read.

4. Is the framework used consistently without unnecessary inline CSS?
Answer: Yes. The project follows a consistent CSS design across different pages. Common styles are kept in the external CSS file, which reduces unnecessary inline CSS and makes the website easier to maintain.

B. Implementation

For the StudentHub Portal, responsive layouts were created for the Home, Registration, Dashboard, and Events pages using HTML and CSS.

- CSS Flexbox was used for the navbar, header, buttons, forms, sidebar, and aligning different elements.
- CSS Grid was used for dashboard cards, feature cards, and event cards.
- A mobile-first approach was followed so the website works properly on mobile, tablet, and desktop screens.
- Media queries were used to change the number of columns, spacing, and alignment according to screen size.

- A common CSS file was used to keep the same colors, fonts, buttons, cards, and overall design on all pages.

Code snippets:

CSS custom variables.

```
:root {
   --primary: steelblue;
   --background: #f5f5f5;
}
body {
   background-color: var(--background);
}
h1 {
   color: var(--primary);
}
```

Button :

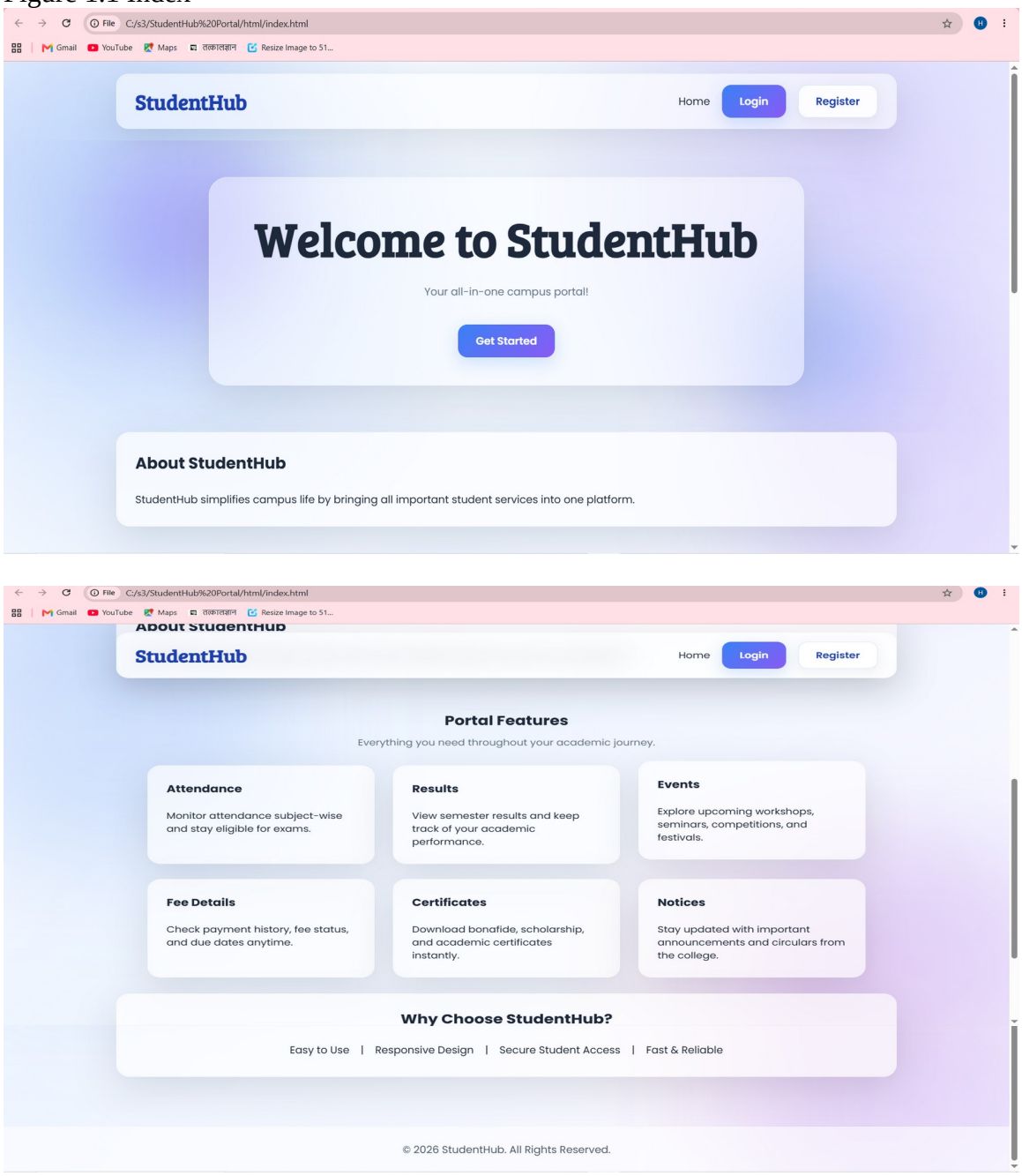
```
button {
   background-color: steelblue;
   color: white;
   padding: 10px 18px;
   border: none;
   border-radius: 6px;
}
button:hover {
   background-color: darkblue;
   cursor: pointer;
```

}

This helped make the Portal responsive, organized, and easy to use on different screen sizes.

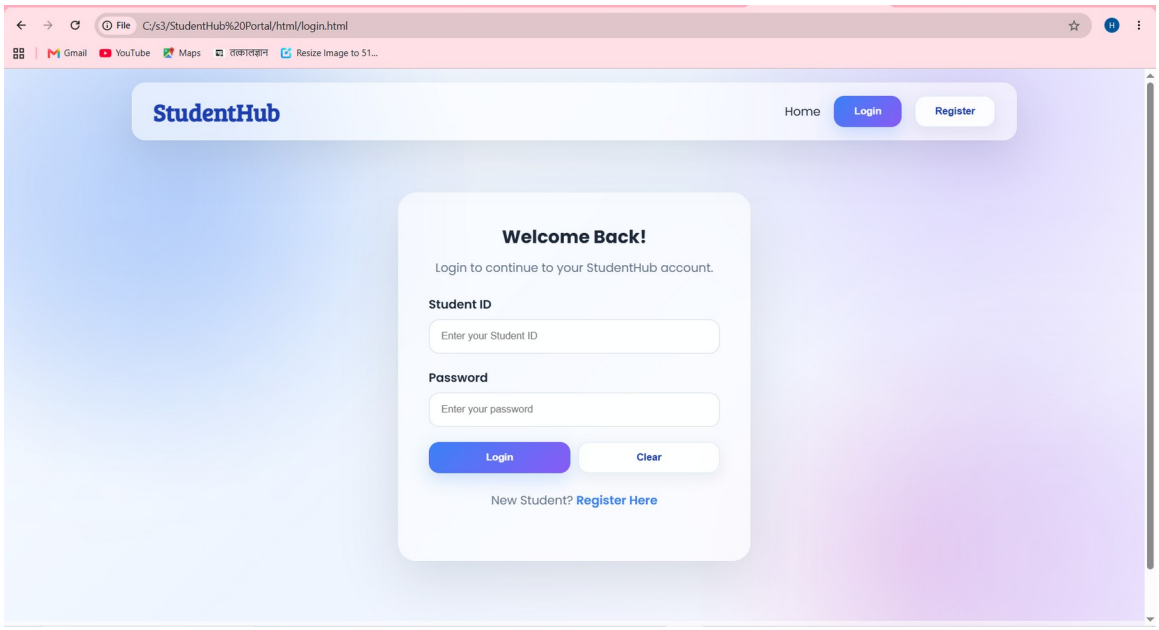
C. Evidence of Work (Screenshots)

Figure 1.1 Index



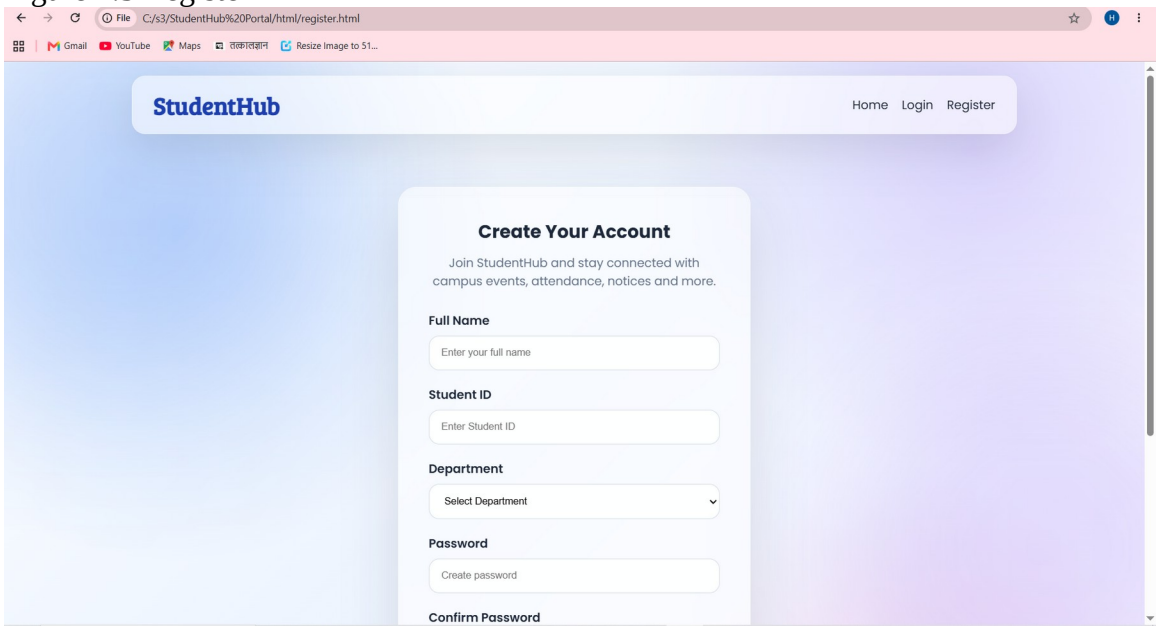
Description: This is home page when user opens website link

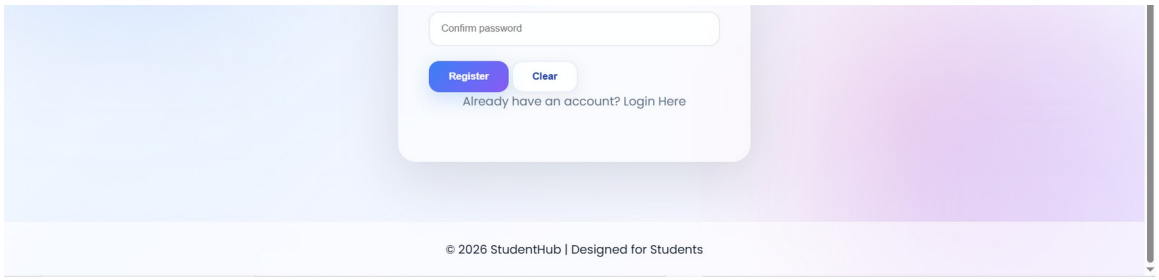
Figure 1.2 Login



Description: This is login page for registered user

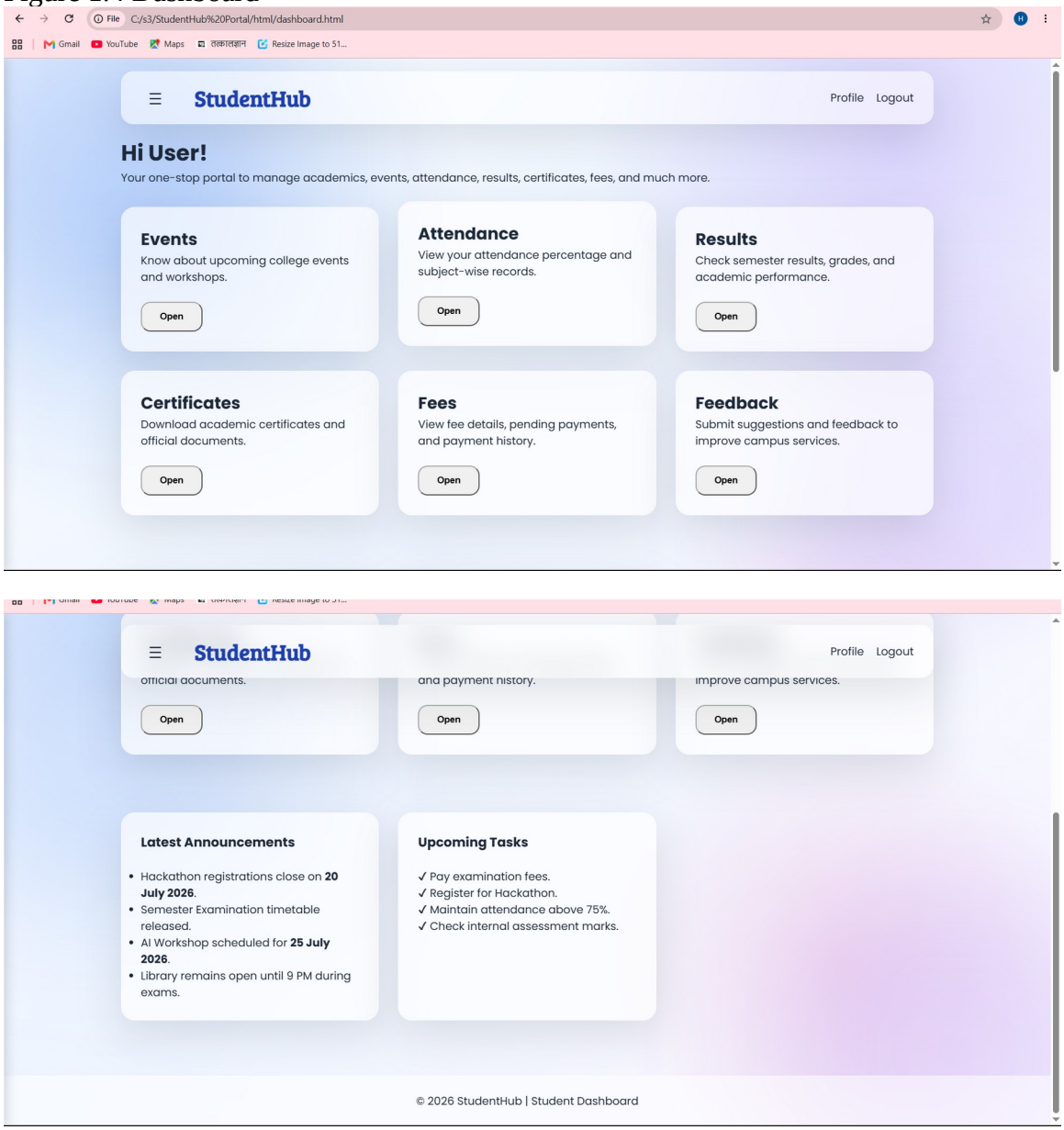
Figure 1.3 Register





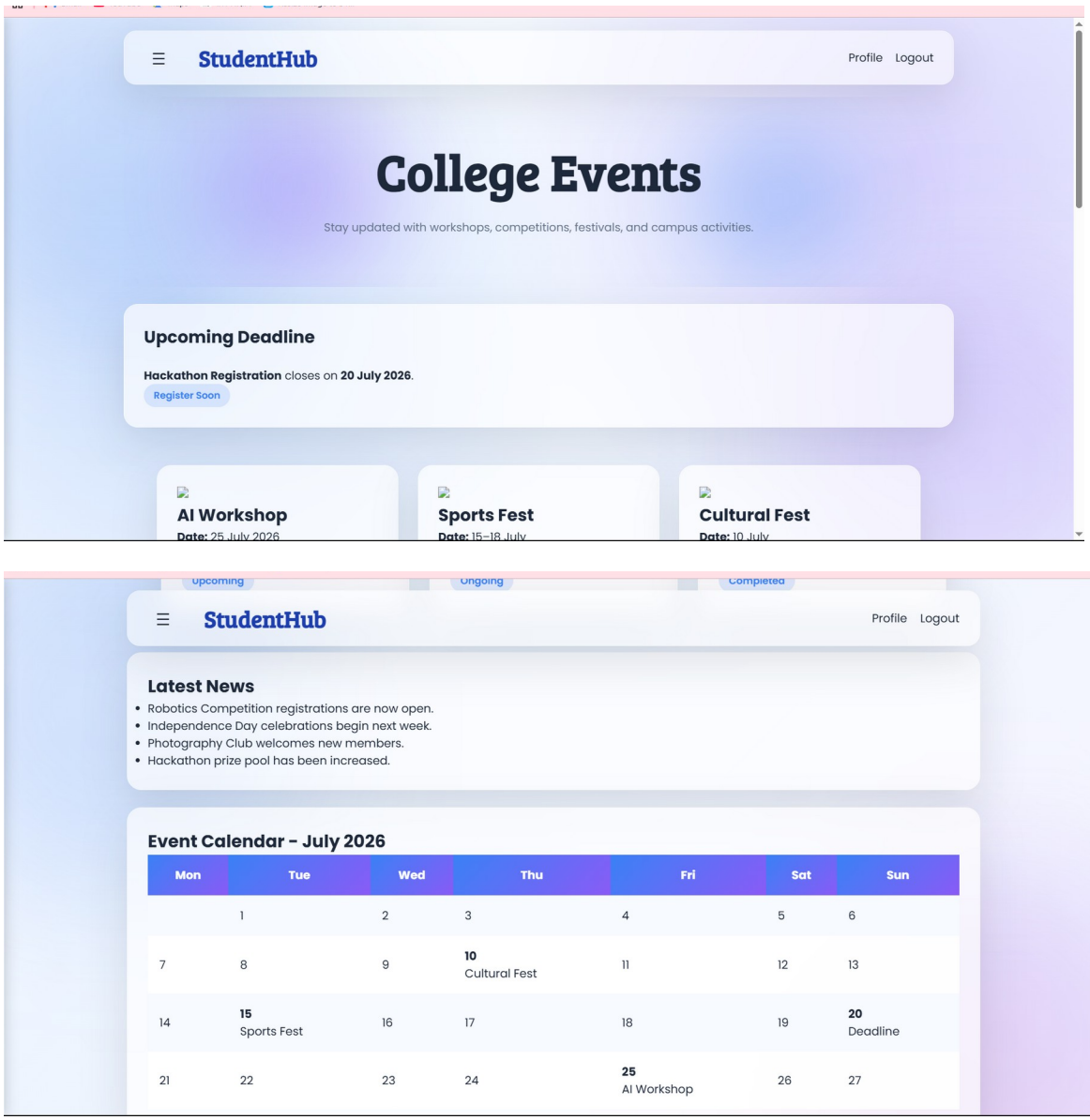
Description: This is register page for new users

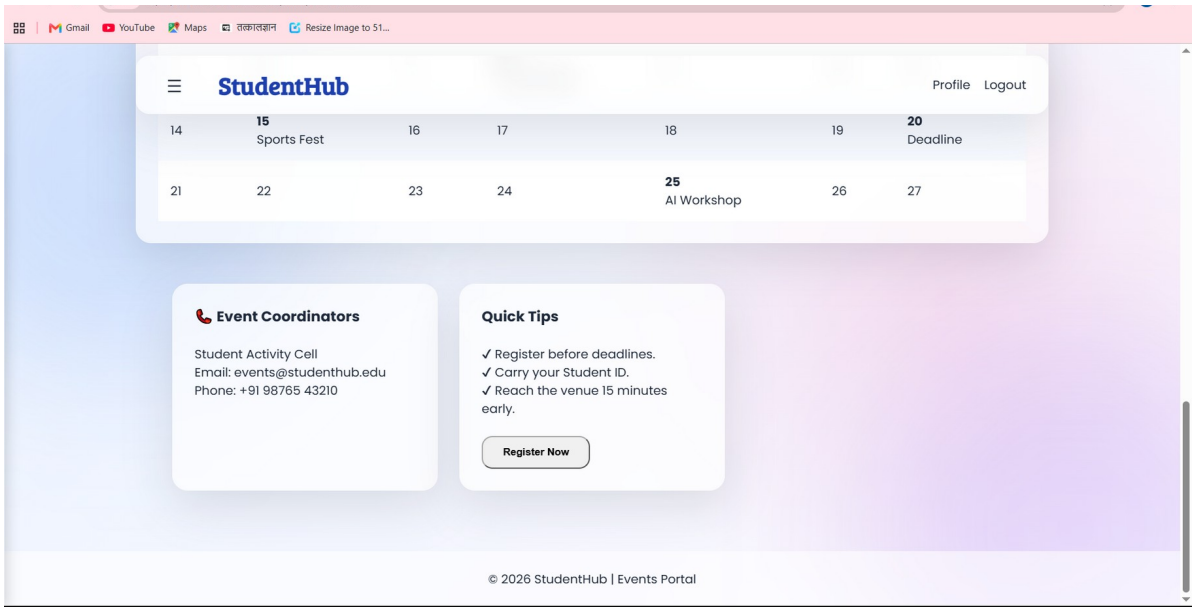
Figure 1.4 Dashboard



Description: This is a dashboard when a user will log in.

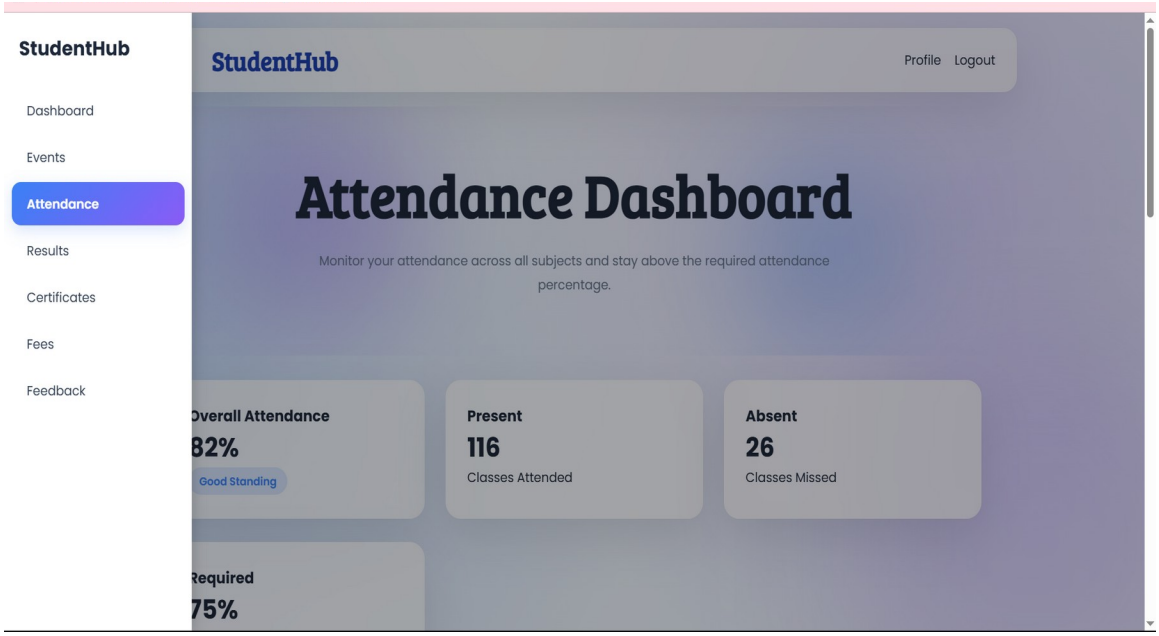
Figure 1.5 Event

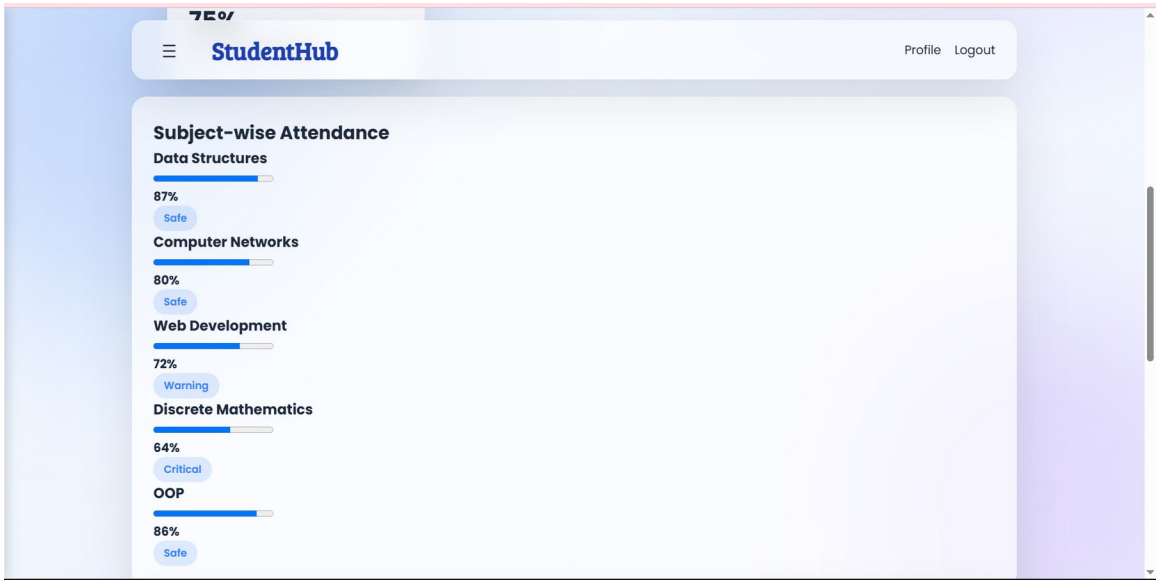


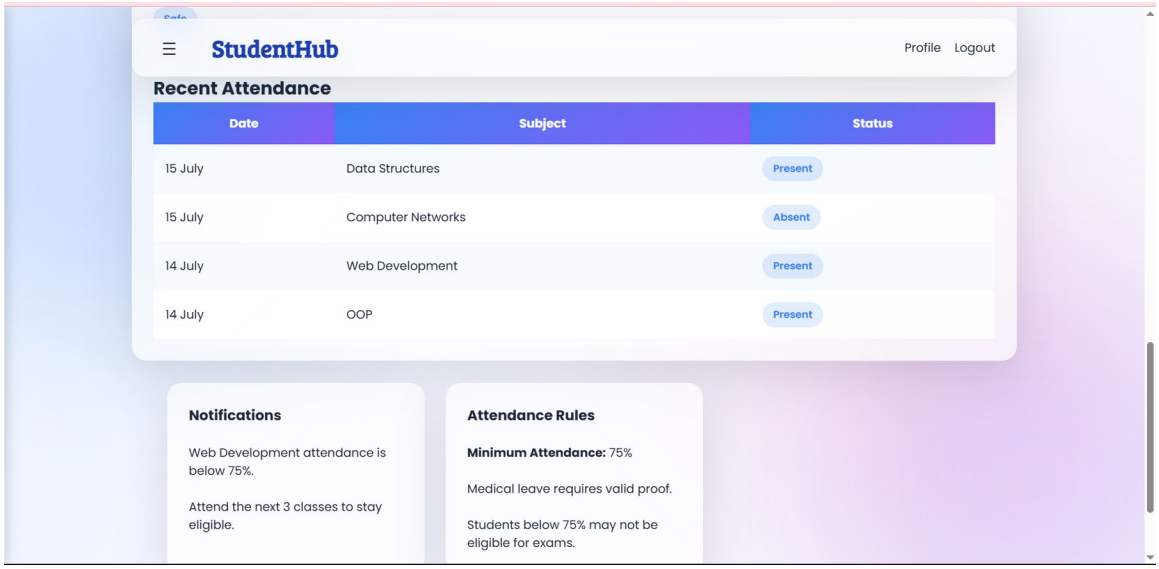


Description: Event page for latest events details

Figure 1.6 Attendance Page

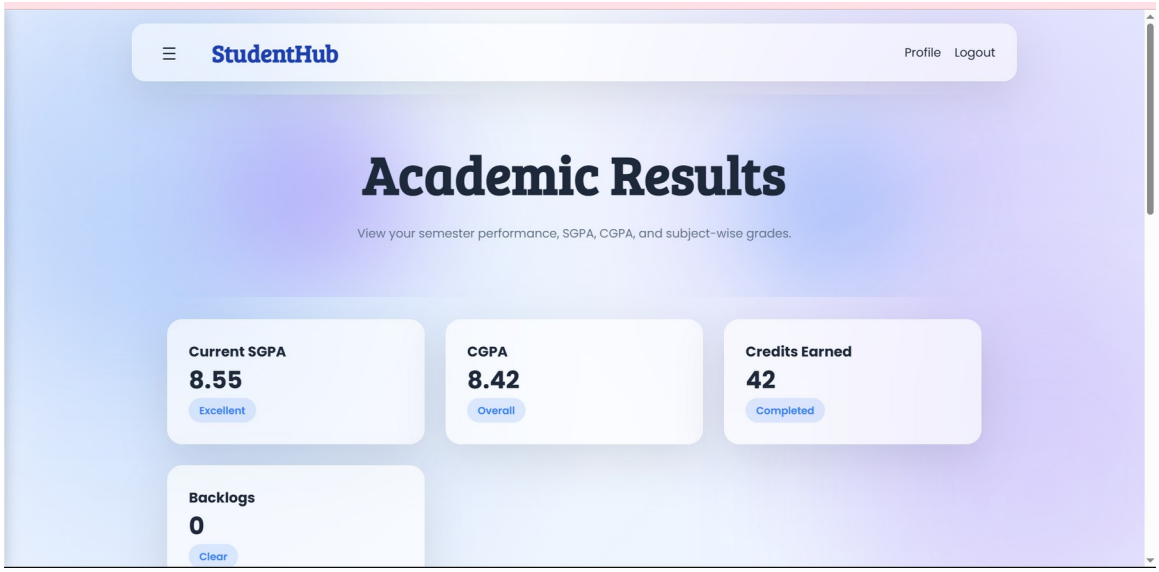

StudentHub
Dashboard
Events
Attendance
Results
Certificates
Fees
Feedback
StudentHub
Profile
Logout
Attendance Dashboard
Monitor your attendance across all subjects and stay above the required attendance percentage.
Overall Attendance
82%
Good Standing
Present
116
Classes Attended
Absent
26
Classes Missed
Required
75%

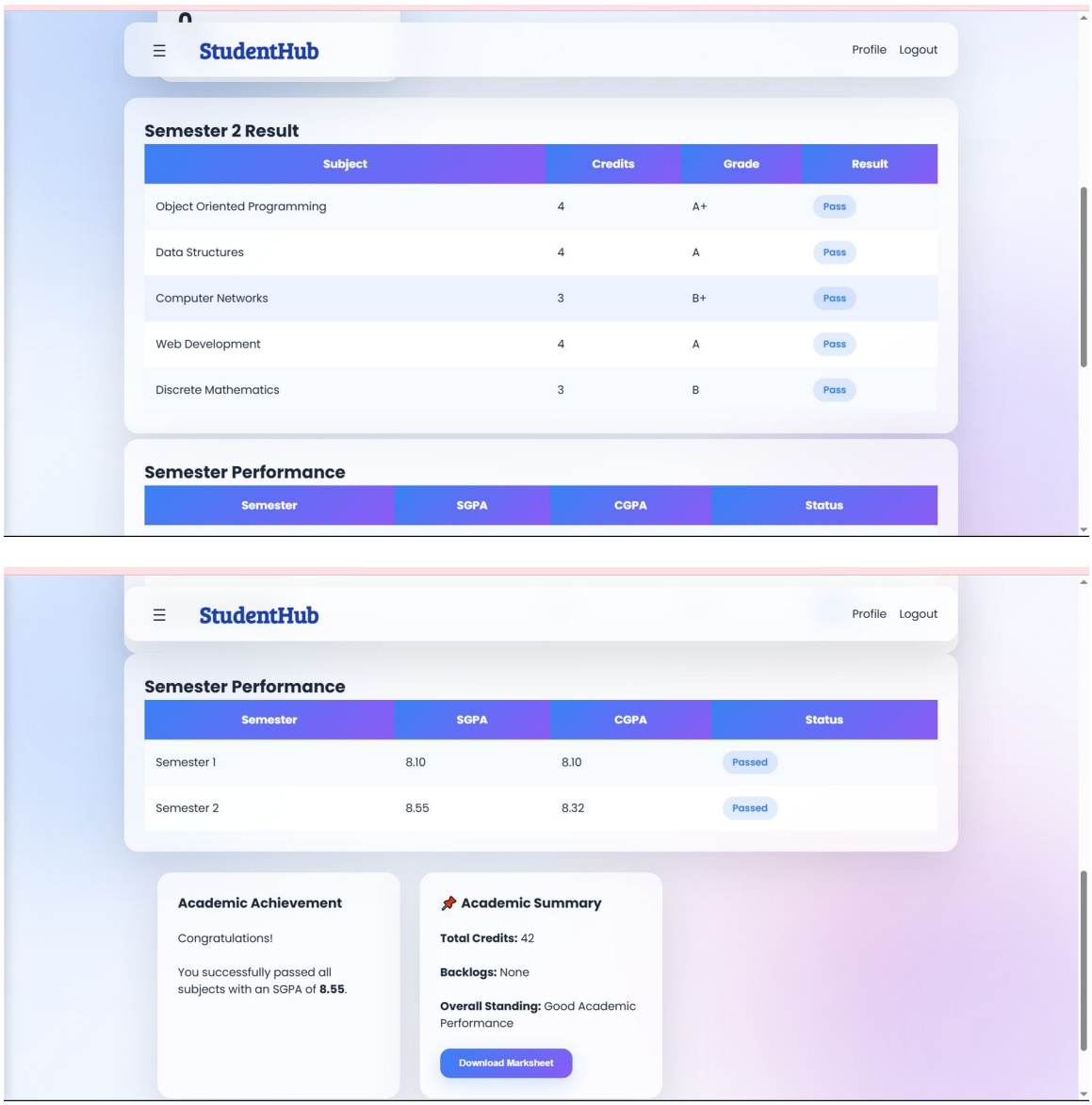

StudentHub
Profile
Logout
Subject-wise Attendance
Data Structures
87%
Safe
Computer Networks
80%
Safe
Web Development
72%
Warning
Discrete Mathematics
64%
Critical
OOP
86%
Safe



Description: This is attendance reports page for user

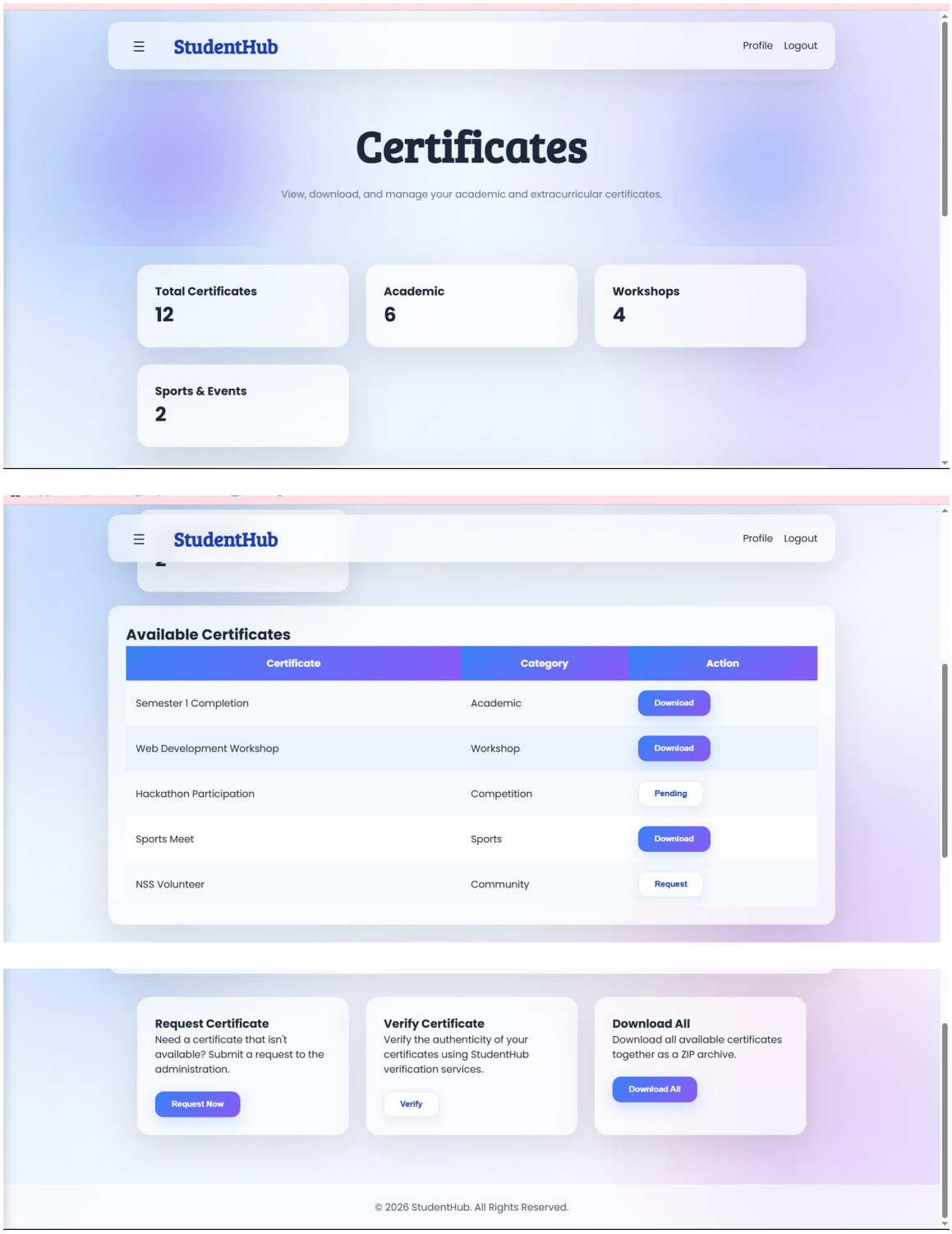
Figure 1.7 Results





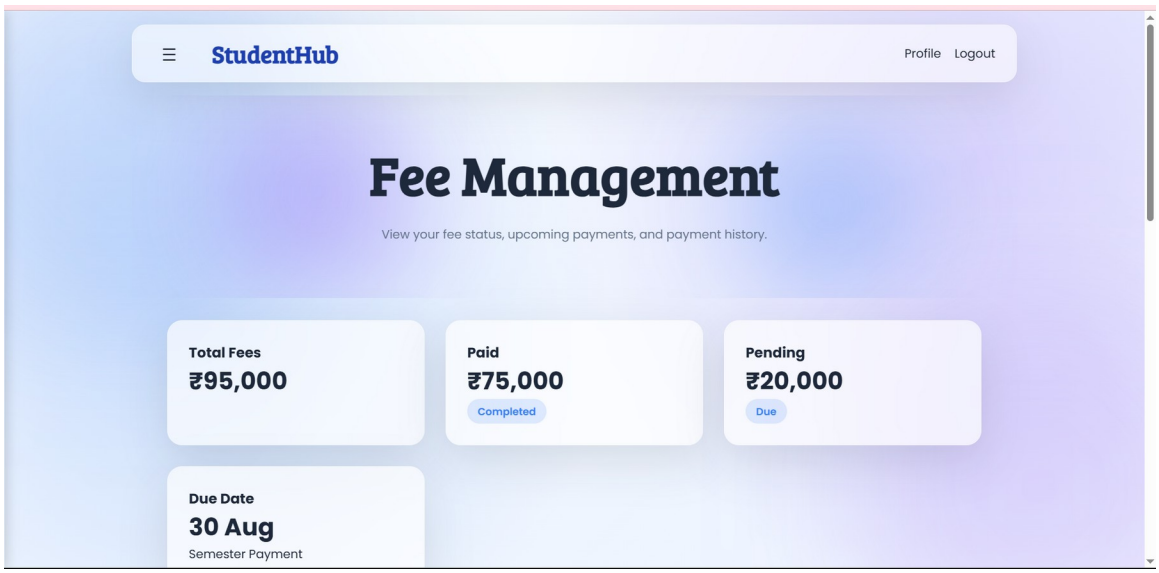
Description: This is result section for user

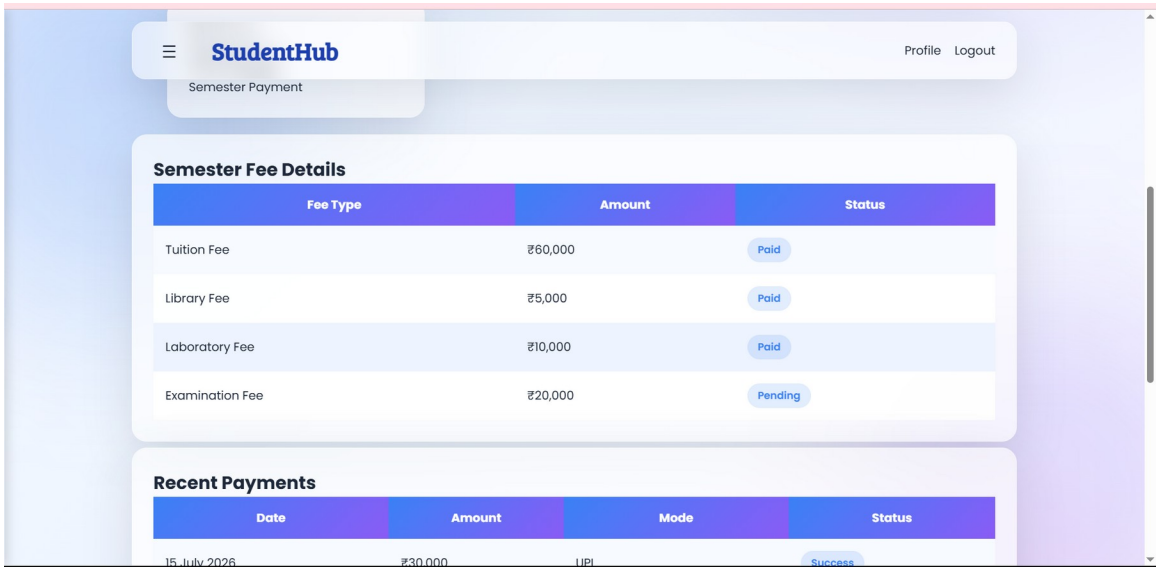
Figure 1.8 Certificate

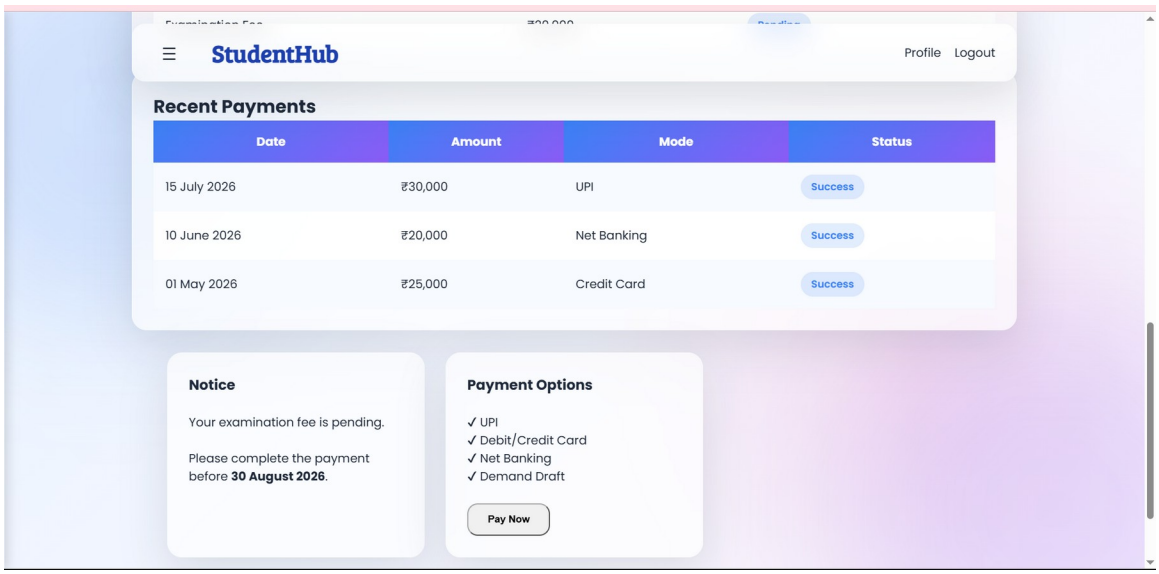


Description: This is certificate section for user

Figure 1.9 Fees

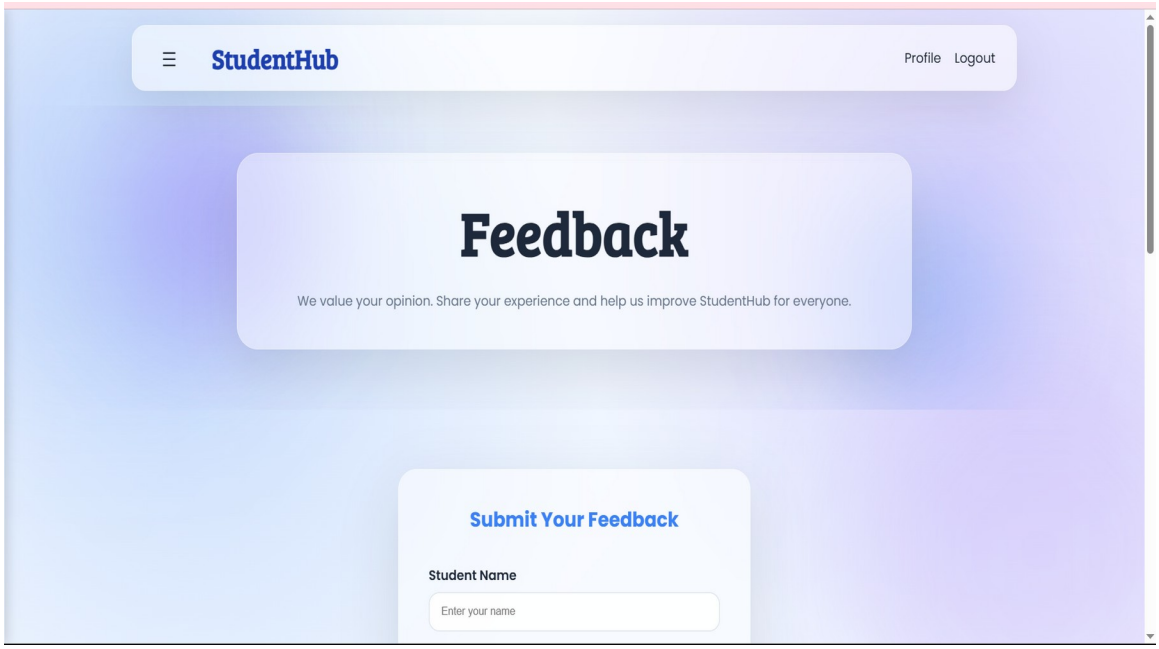

StudentHub
Profile
Logout
Fee Management
View your fee status, upcoming payments, and payment history.
Total Fees
₹95,000
Paid
₹75,000
Completed
Pending
₹20,000
Due
Due Date
30 Aug
Semester Payment

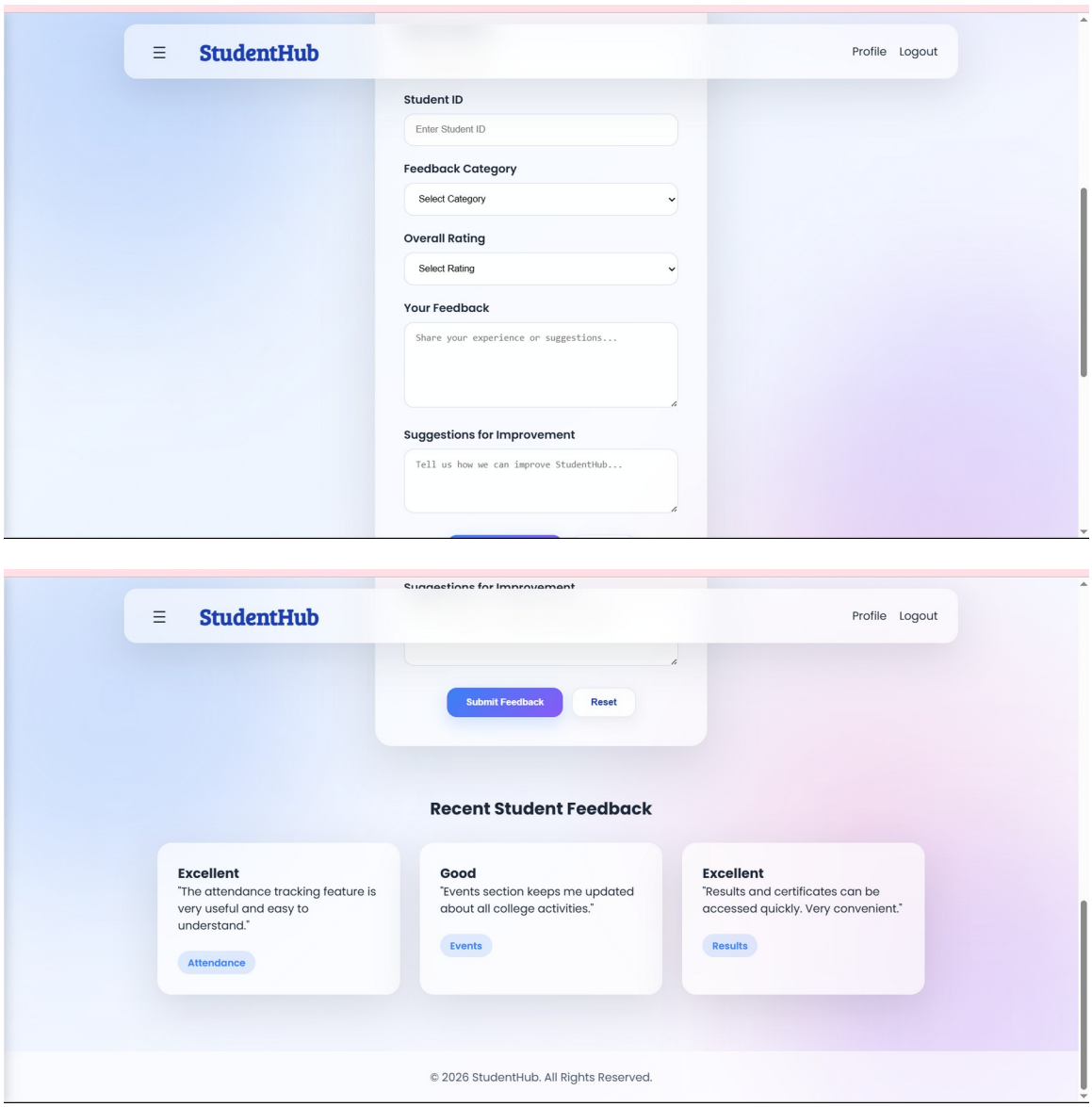

StudentHub
Profile
Logout
Semester Payment
Semester Fee Details
Fee Type
Amount
Status
Tuition Fee
₹60,000
Paid
Library Fee
₹5,000
Paid
Laboratory Fee
₹10,000
Paid
Examination Fee
₹20,000
Pending
Recent Payments
Date
Amount
Mode
Status
15 July 2026
₹30,000
UPI
Success



Description : This section gives info and direction on fees payment

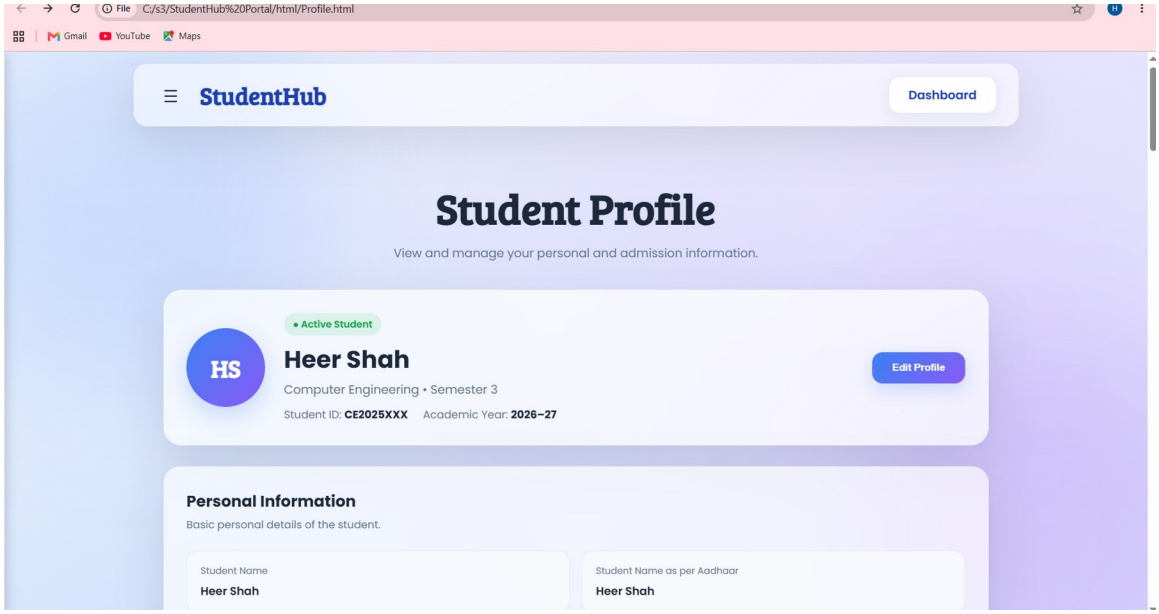
Figure 1.10 Feedback

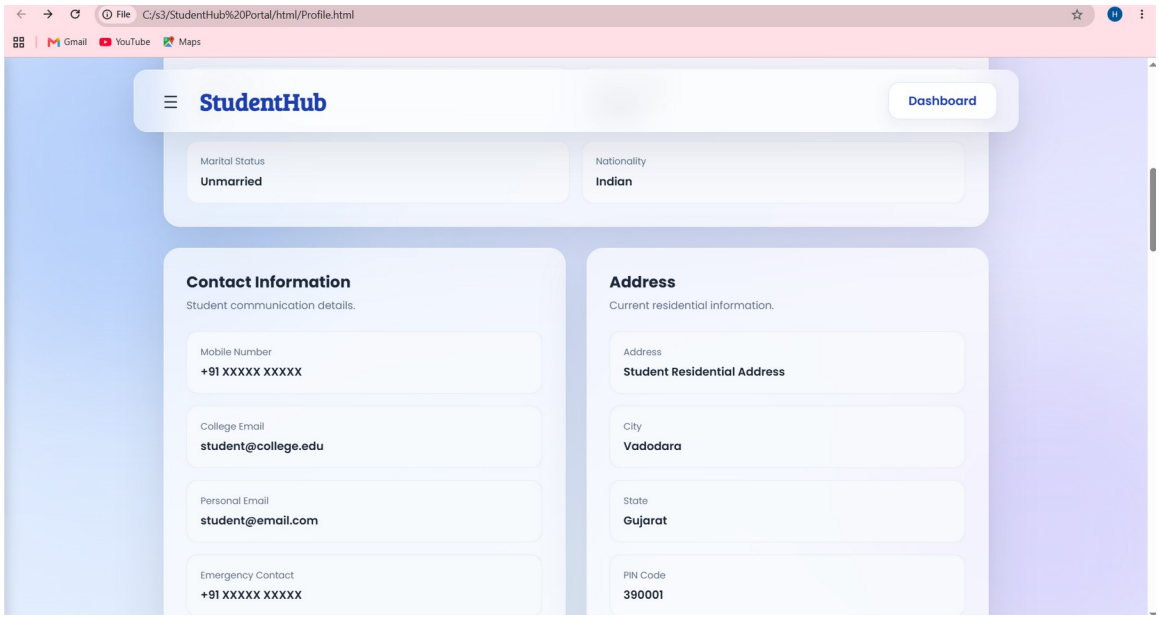


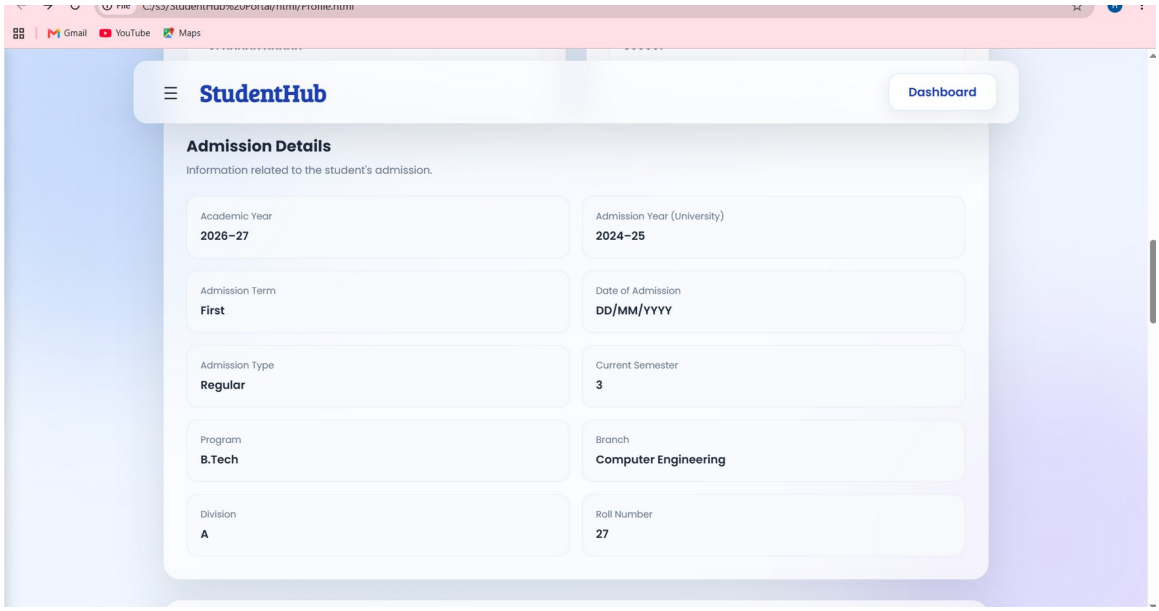


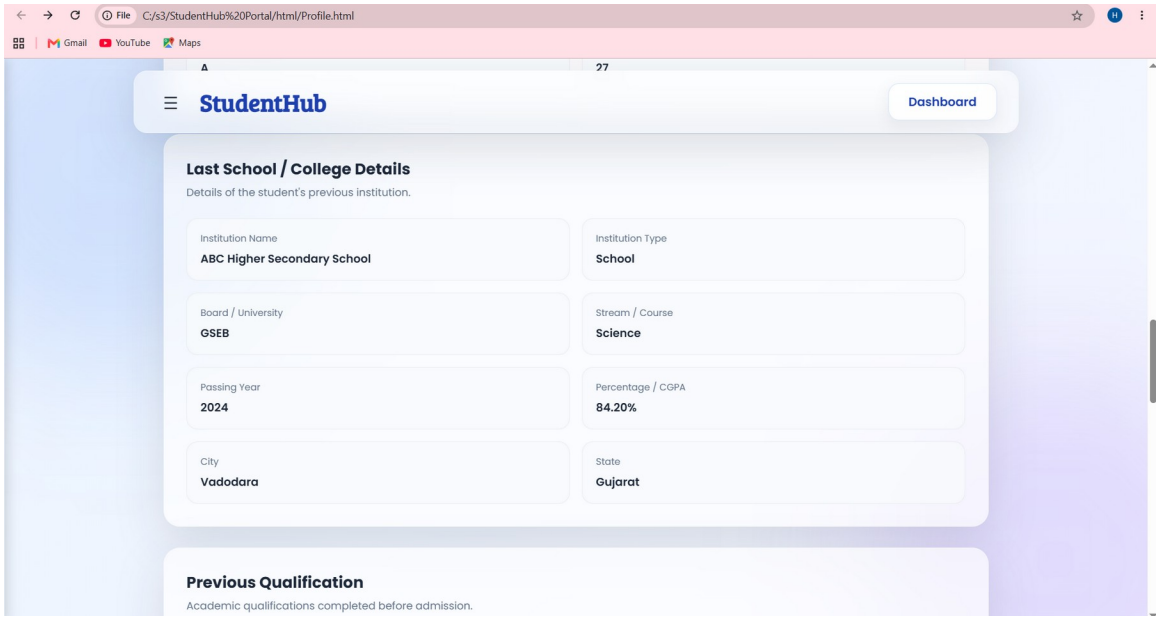
Description: This section helps in keeping up with users feedback on website

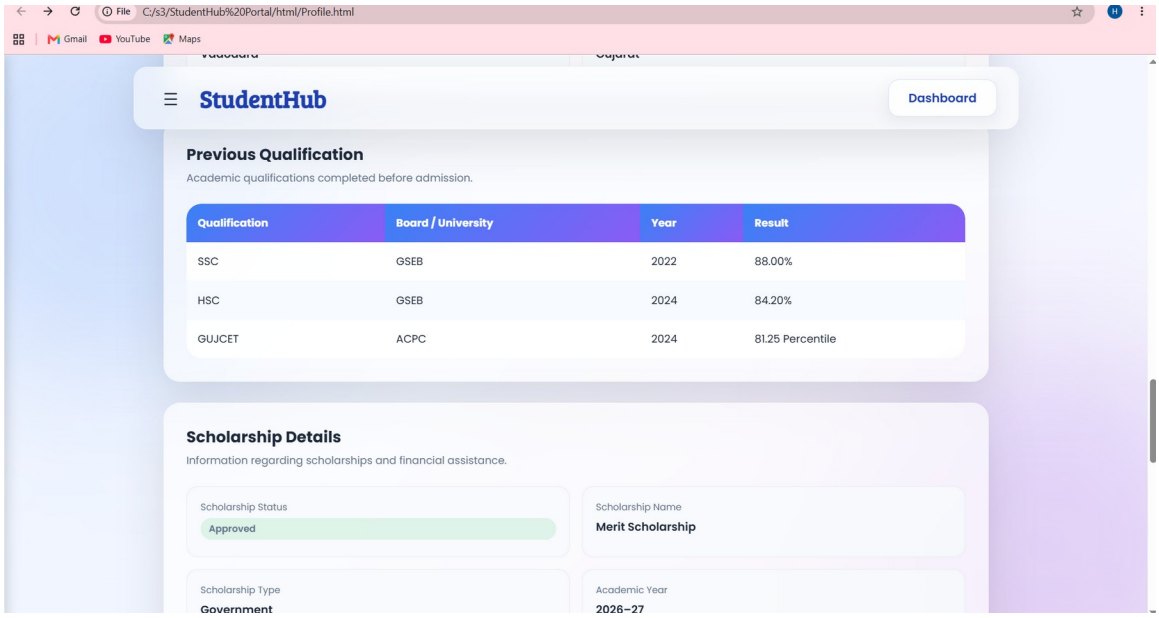
Figure 1.11 Profile

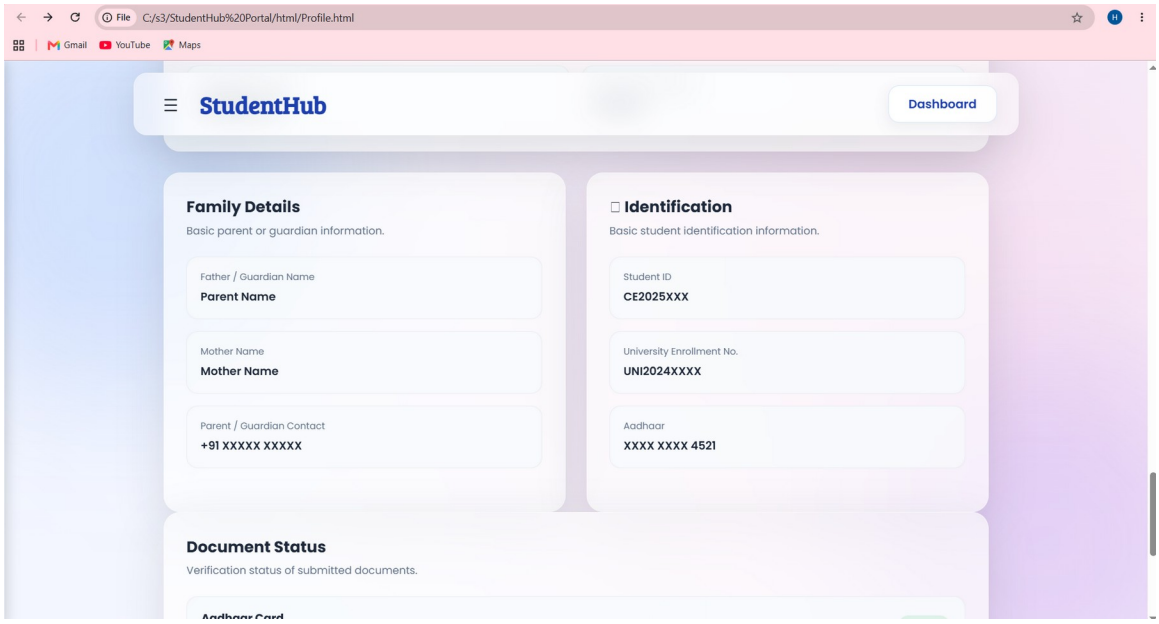


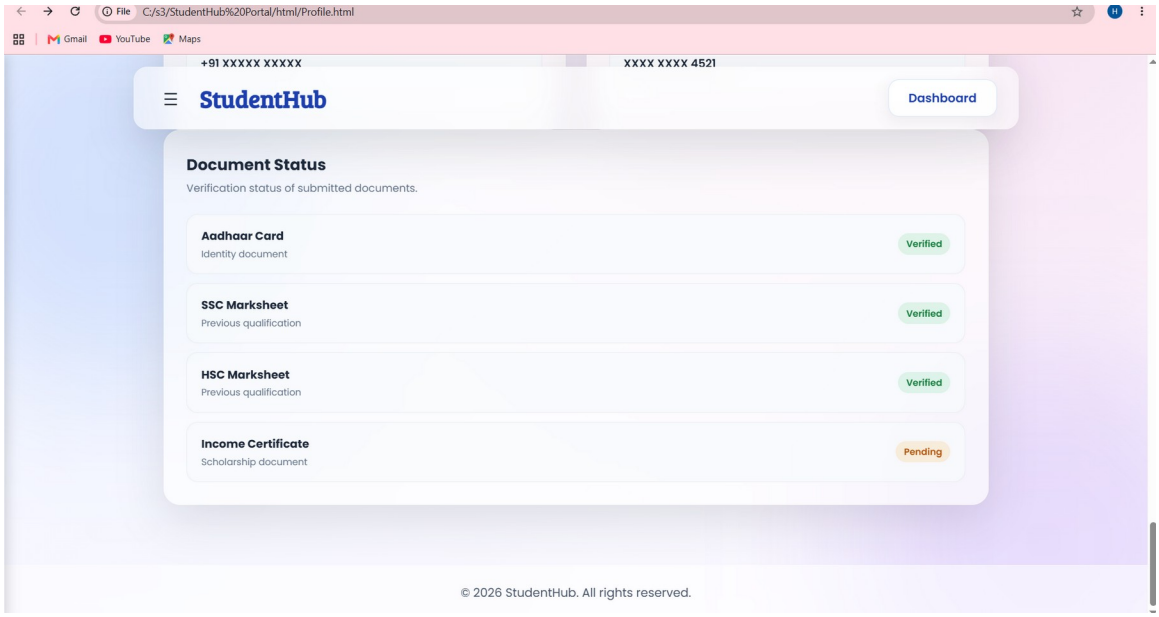



Gmail
YouTube
Maps
StudentHub
Dashboard
Admission Details
Information related to the student's admission.
Academic Year
2026–27
Admission Year (University)
2024–25
Admission Term
First
Date of Admission
DD/MM/YYYY
Admission Type
Regular
Current Semester
3
Program
B.Tech
Branch
Computer Engineering
Division
A
Roll Number
27


File
C:/s3/StudentHub%20Portal/html/Profile.html
Gmail
YouTube
Maps
StudentHub
Dashboard
Last School / College Details
Details of the student's previous institution.
Institution Name
ABC Higher Secondary School
Institution Type
School
Board / University
GSEB
Stream / Course
Science
Passing Year
2024
Percentage / CGPA
84.20%
City
Vadodara
State
Gujarat
Previous Qualification
Academic qualifications completed before admission.


File C:/s3/StudentHub%20Portal/html/Profile.html
Gmail YouTube Maps
StudentHub
Dashboard
Previous Qualification
Academic qualifications completed before admission.
Qualification
Board / University
Year
Result
SSC
GSEB
2022
88.00%
HSC
GSEB
2024
84.20%
GUJCET
ACPC
2024
81.25 Percentile
Scholarship Details
Information regarding scholarships and financial assistance.
Scholarship Status
Approved
Scholarship Name
Merit Scholarship
Scholarship Type
Government
Academic Year
2026–27


File C:/s3/StudentHub%20Portal/html/Profile.html
Gmail YouTube Maps
StudentHub
Dashboard
Family Details
Basic parent or guardian information.
Father / Guardian Name
Parent Name
Mother Name
Mother Name
Parent / Guardian Contact
+91 XXXXX XXXXX
Identification
Basic student identification information.
Student ID
CE2025XXX
University Enrollment No.
UNI2024XXXX
Aadhaar
XXXX XXXX 4521
Document Status
Verification status of submitted documents.



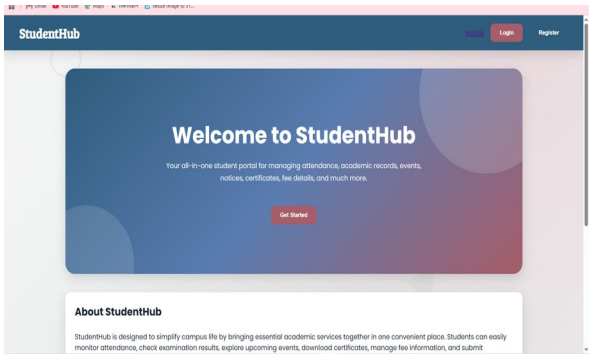
Description: This page is a profile page containing information about the user.

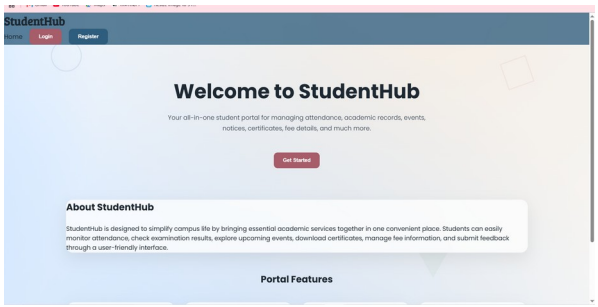
D. GitHub Repository Link:

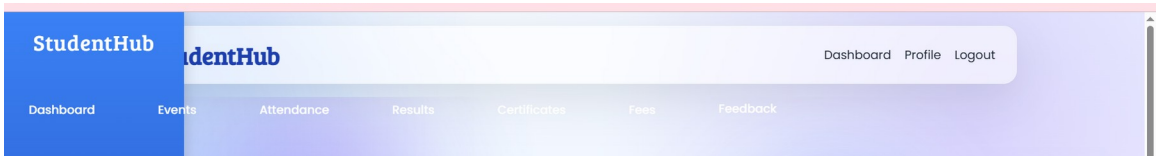
https://github.com/heerce27/StudentHub-Portal.git

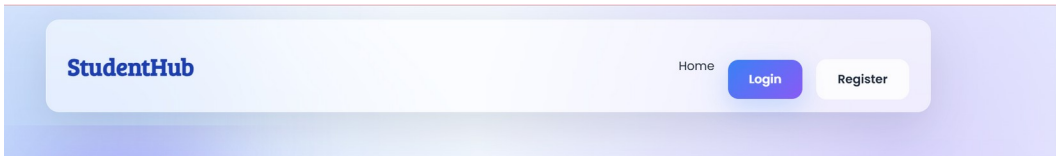
E. Challenges Faced:

I faced many challenges while designing this style.css like fixing button alignment and spacing on different pages, Preventing text and content from spreading too much horizontally, Managing card widths and alignment, as some cards were stretching across the page, Making the sidebar and menu button work properly without affecting the existing layout.

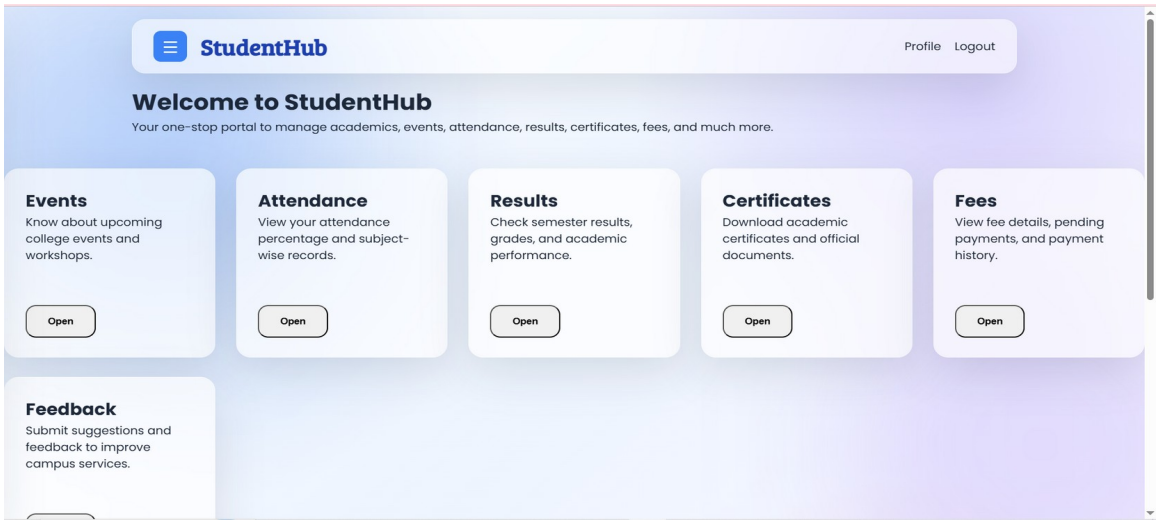












F. Learning Outcomes:

While creating this css i learned

- How to use CSS Flexbox and Grid for arranging website elements.
- How to create responsive layouts using media queries.
- How to choose and maintain a consistent colour, font, spacing, and button style.
- How to fix common layout problems like overlapping elements, stretched cards, and alignment issues.
- How to organize CSS so that the same styling can be reused across different pages.
- I also got better at debugging CSS and improving a design step by step instead of changing everything at once.

Faculty Remarks:

Practical - 4

Practical Title: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

A. Theory

1. How are DOM elements selected and modified?

Answer: DOM elements are selected using methods like getElementById() and querySelector(), and modified using properties like textContent, style, and classList

2. Are event listeners attached correctly?

Answer: Yes, event listeners are attached correctly using addEventListener() to handle user actions like clicks.

3. Is localStorage used for remembering theme choice?

Answer: Yes, localStorage stores the user's theme choice so it remains the same even after refreshing the page.

4.Does interactivity improve usability without breaking accessibility?

Answer: Yes, interactivity makes the website easier to use while keeping buttons, navigation, and content accessible.

B. Implementation

FAQ: Questions can be expanded and collapsed by clicking on them.

Modal Popup: A popup window appears when the user clicks the required button and can be closed using the close button.

Slider: Users can change between different content using navigation controls.

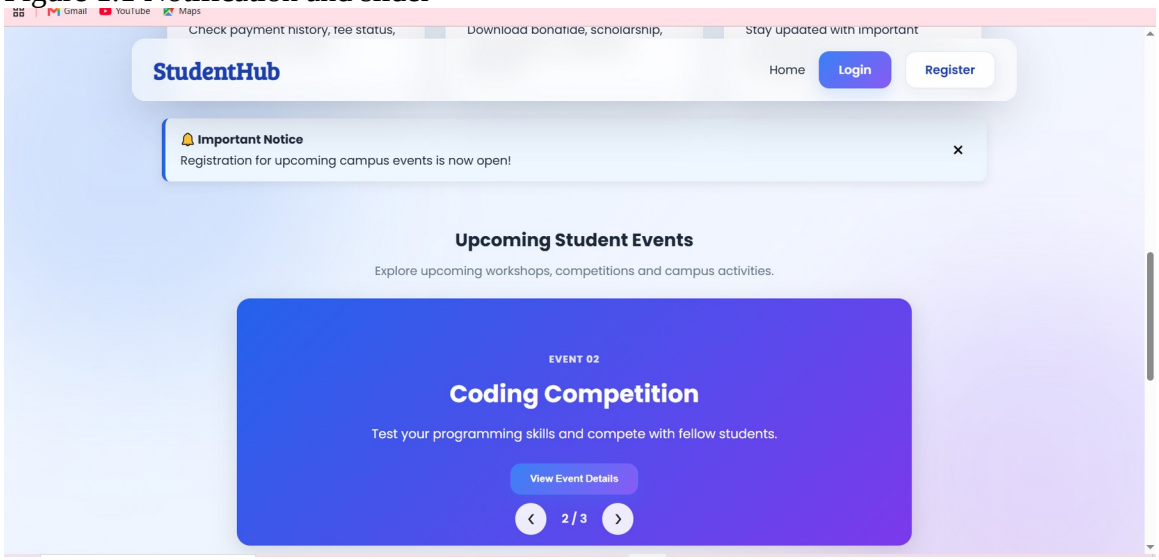
Notification Banner: A notification message is displayed and can be closed by the user.

Hamburger Menu: The navigation menu opens and closes when the hamburger icon is clicked.

Light/Dark Theme: JavaScript changes the theme and uses localStorage to remember the user's selected theme after refreshing the page.

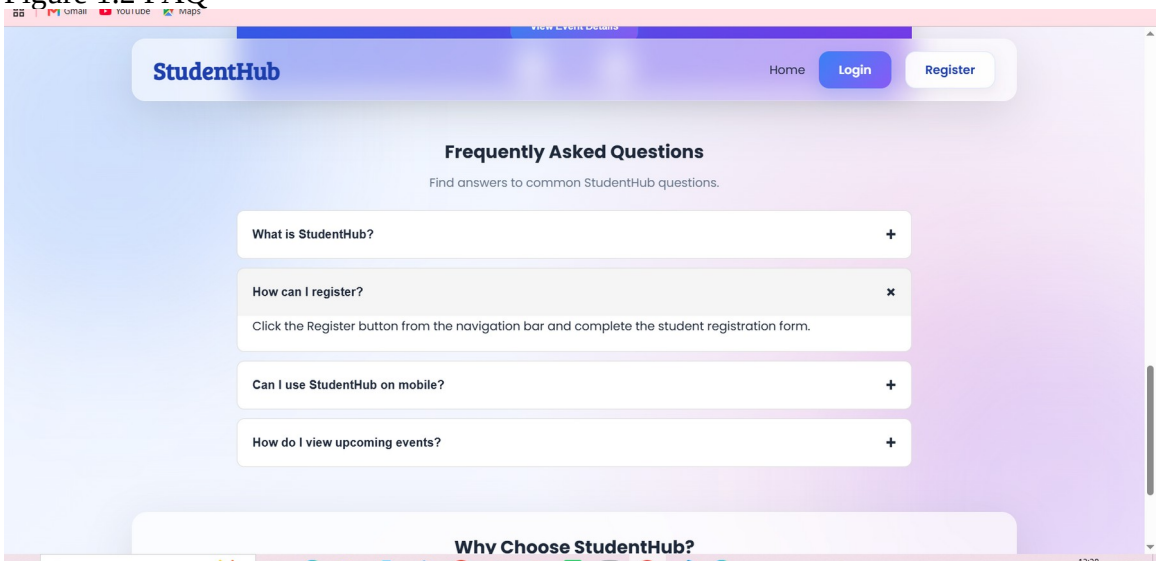
C. Evidence of Work (Screenshots)

Figure 1.1 Notification and slider



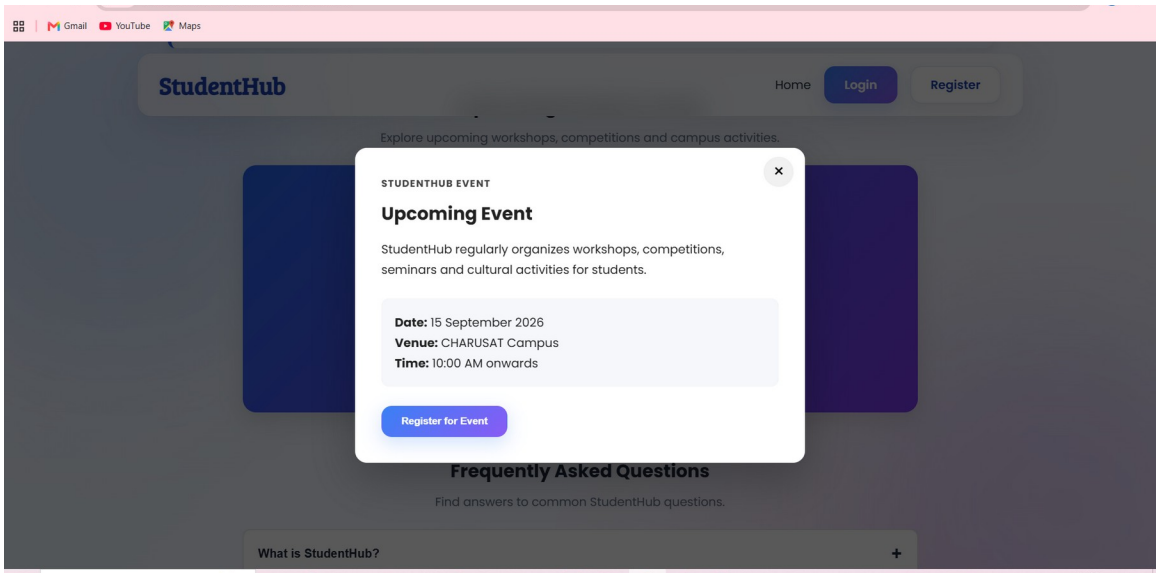
Description: Users can change between different content and remove appeared notification

Figure 1.2 FAQ



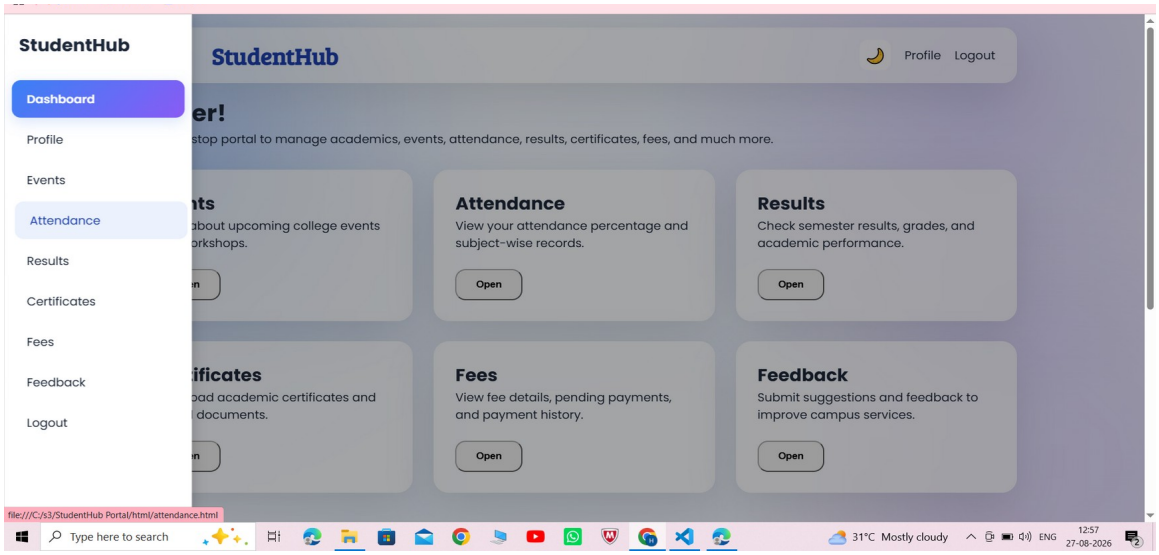
Description: users can click on questions and related answers will be shown.

Figure 1.3 Modal pop up



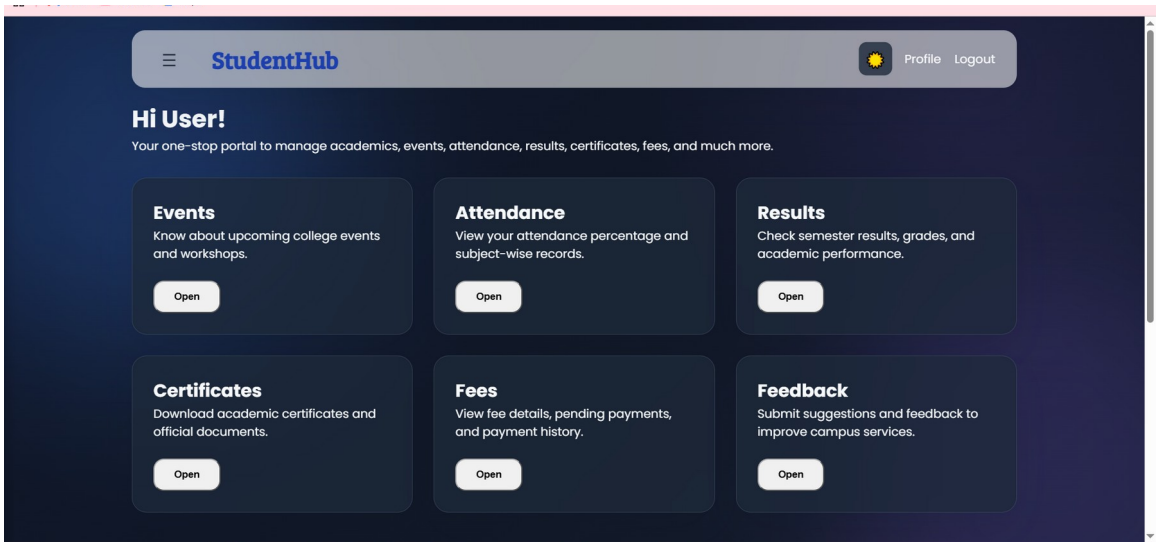
Description: Notification banner is displayed when user clicks on button and can be closed by clicking on cross.

Figure 1.4 Slidebar



Description: Slidebar when clicked on menu button. It stays open even when the page is changed. After clicking on the outside part of the slidebar, it closes.

Figure 1.5 Light/Dark theme



Description: Dark theme is applied on the website pages when the user clicks on the button .

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

Faced challenges like:

1. Slidebar application on every page was difficult as I wanted it to stay open even after changing the page.
2. Remembering the user's choice of dark theme and storing it in local storage.
3. Applying sliding content on the index page and maintaining the measurements.
4. Making sure all interactive components worked properly without affecting the overall webpage design.

F. Learning Outcomes:

1. Learned how to use DOM manipulation to dynamically change webpage elements.
2. Learned how to handle user actions using event listeners.
3. Learned to create interactive components like FAQ, modal popup, slider, notification banner, and hamburger menu.
4. Learned how to use localStorage to remember the user's theme preference.
5. Learned how to combine HTML, CSS, and JavaScript to create a more interactive and user-friendly webpage.
6. Improved my understanding of responsive design and maintaining proper layout measurements.

Faculty Remarks:

Practical - 5

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory

1. Are correct input types and attributes used?

Answer: Yes, suitable input types like text,email and password are used with attributes such as required and placeholder for proper input and validation.

2. Are validation errors displayed near the relevant fields?

Answer: Yes, validation error messages are displayed close to the corresponding input field, making it easy for the user to identify and correct the mistake.

3. Is password strength checked?

Answer: Yes, password strength is checked using conditions such as minimum length and a combination of uppercase, lowercase, numbers, and special characters.

4. Is the form accessible using keyboard and screen-reader-friendly labels?

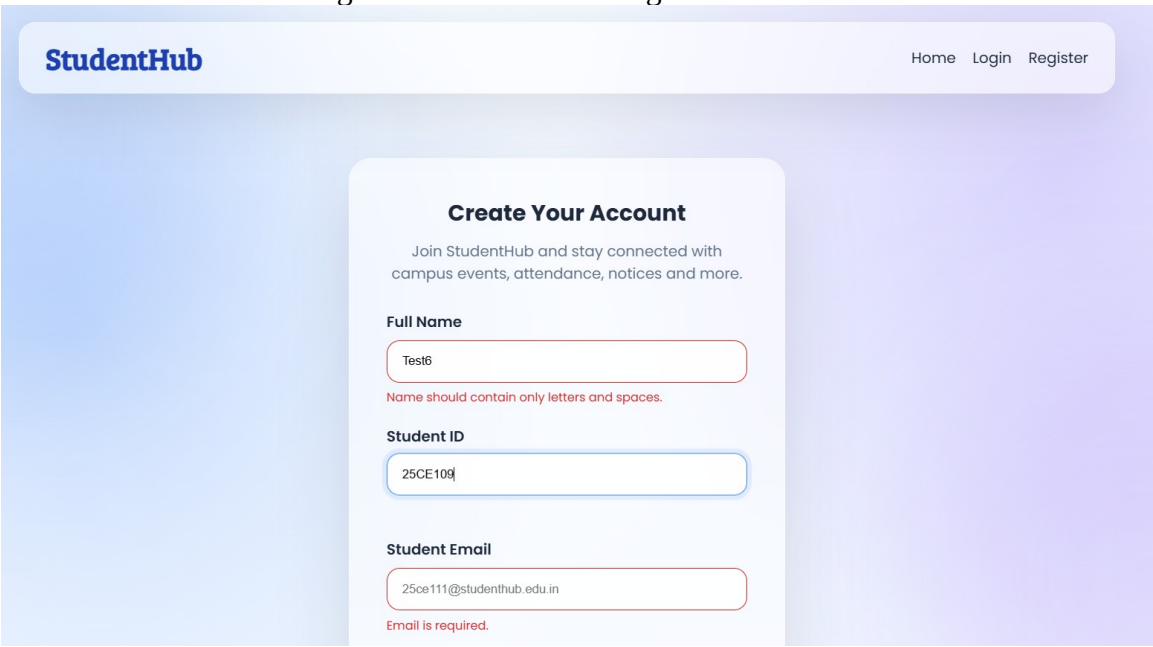
Answer: Yes, the form can be navigated using the keyboard and proper <label> elements are associated with input fields to support screen readers.

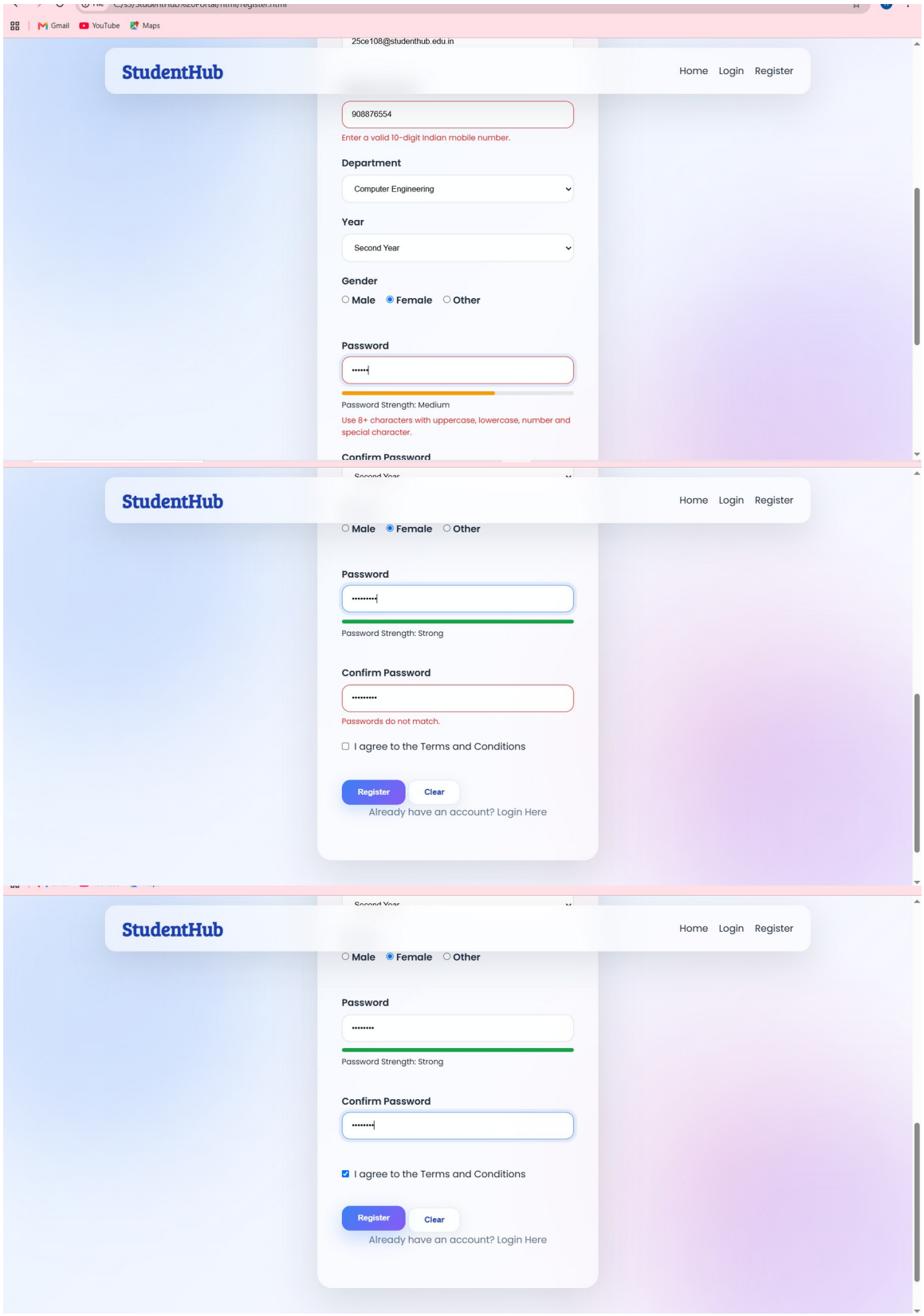
B. Implementation

- Added input fields for student name, email, mobile number, password and confirm password.
- Added course, year and gender selection fields along with the terms and conditions checkbox.
- Used suitable HTML5 input types such as text, email and password.
- Used JavaScript validation to check whether the entered details are correct.
- Used Regex for validating the name, email and mobile number.
- Added password strength checking and checked whether the confirm password matches the password.
- Displayed validation messages near the fields when incorrect data was entered.
- Tested the form with both invalid and valid inputs.
- After entering all details correctly and accepting the terms, the form allows successful submission.

C. Evidence of Work (Screenshots)

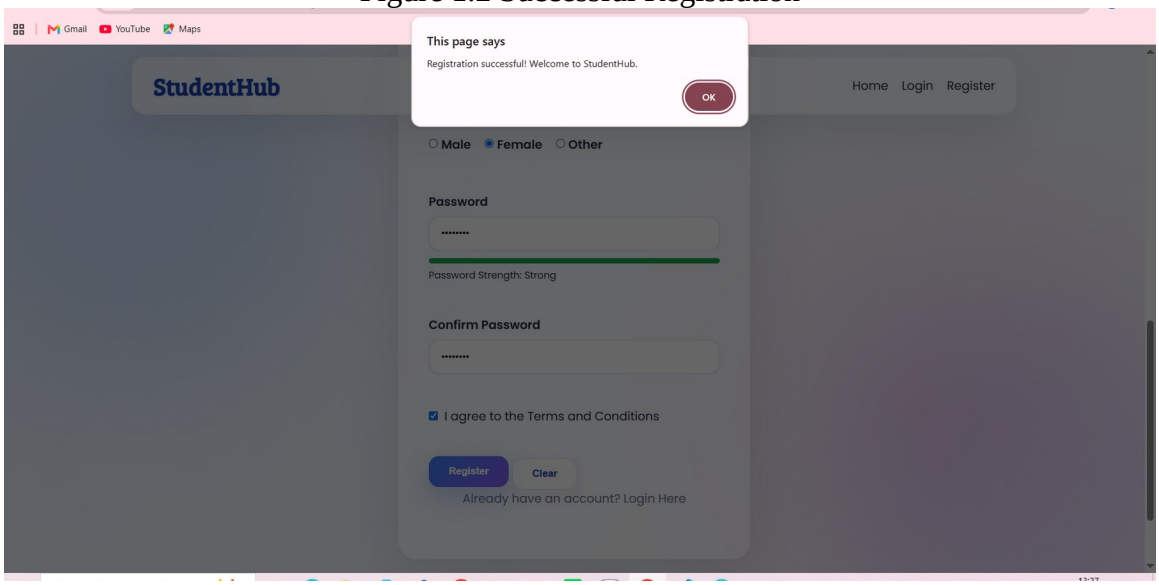
Figure 1.1 StudentHub Registration Form




Gmail
YouTube
Maps
25ce108@studenthub.edu.in
StudentHub
Home
Login
Register
908876554
Enter a valid 10-digit Indian mobile number.
Department
Computer Engineering
Year
Second Year
Gender
Male
Female
Other
Password
Password Strength: Medium
Use 8+ characters with uppercase, lowercase, number and special character.
Confirm Password
StudentHub
Home
Login
Register
Male
Female
Other
Password
Password Strength: Strong
Confirm Password
Passwords do not match.
I agree to the Terms and Conditions
Register
Clear
Already have an account? Login Here
StudentHub
Home
Login
Register
Male
Female
Other
Password
Password Strength: Strong
Confirm Password
I agree to the Terms and Conditions
Register
Clear
Already have an account? Login Here

Description : These screenshots represent the registration page . They validate the name,phone number, student id, mail and password strength and password matching.

Figure 1.2 Successful Registration



Description: and then if all fields are correctly filled then shows successful registration.

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

- I faced some difficulty in applying JavaScript validation correctly to all the registration fields.
- Writing the Regular Expressions for email and mobile number validation took some trial and error.
- I had to fix the condition for the password and confirm password so that both values matched correctly.
- Initially, some validation messages were not appearing at the correct place, so I adjusted the JavaScript and html structure.

F. Learning Outcomes:

- I learned how to use different input types like email, text and password according to the requirement.
- I learned how to write and apply Regex in JavaScript for form validation.
- I understood how to check password strength and confirm password matching.
- I learned how to display error messages based on the user's input instead of only using HTML validation.

Faculty Remarks:

Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

A. Theory

1. How is JSON fetched, parsed, and rendered?

Answer: JSON data is fetched using the Fetch API with fetch(). The response is converted into JavaScript data using .json(). The data is then processed and displayed dynamically on the webpage using JavaScript, mainly with map() and HTML elements.

2. Which array methods are used for search, filter, sort, and pagination?

Answer:
Search: filter() is used to find records matching the search text.
Filter: filter() is used to display records based on a selected category or condition.
Sort: sort() is used to arrange records in ascending or descending order.
Pagination: slice() is used to display only a fixed number of records on each page.
Rendering: map() is used to convert the data into HTML elements.

3. How are loading and error states handled?

Answer:A loading message is displayed while the JSON data is being fetched. If the data is loaded successfully, the message is removed and the records are displayed. If the fetch fails or the JSON file is missing/invalid, an error message is shown using catch() so that the user knows something went wrong.

4. How is modularity maintained in JavaScript files?

Answer: Modularity is maintained by dividing the code into separate functions, where each function performs one task, such as fetching data, rendering records, searching, filtering, sorting, or pagination. This makes the code easier to understand, test, reuse, and modify.

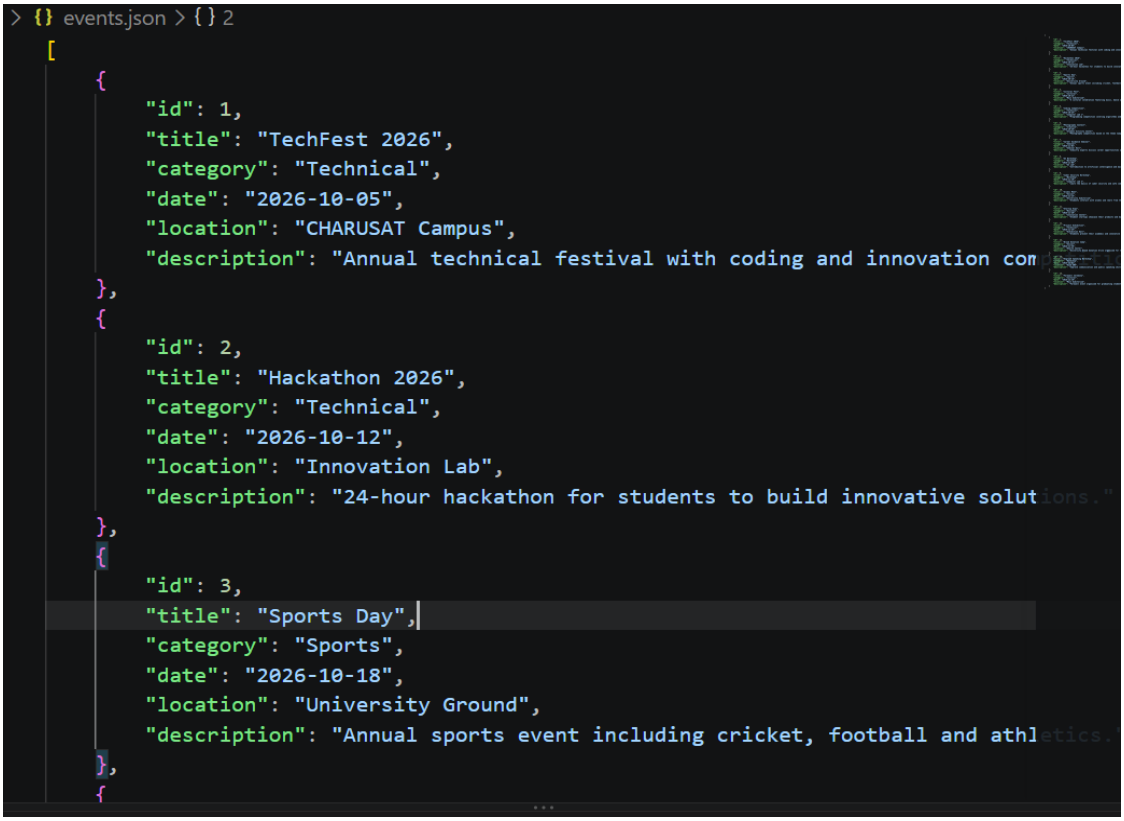
B. Implementation/ Output/Result

- Created external JSON files containing the required records such as events and students.
- Used the JavaScript fetch() method to load the JSON data.
- Parsed the response using .json() and stored the data in JavaScript arrays.
- Used map() to dynamically generate HTML elements and display the records on the webpage.
- Implemented filter() for searching and filtering records based on user input.
- Added loading and error messages to handle data loading and invalid/missing JSON files.

- Tested the application using different search values, filters, pages, and JSON data.

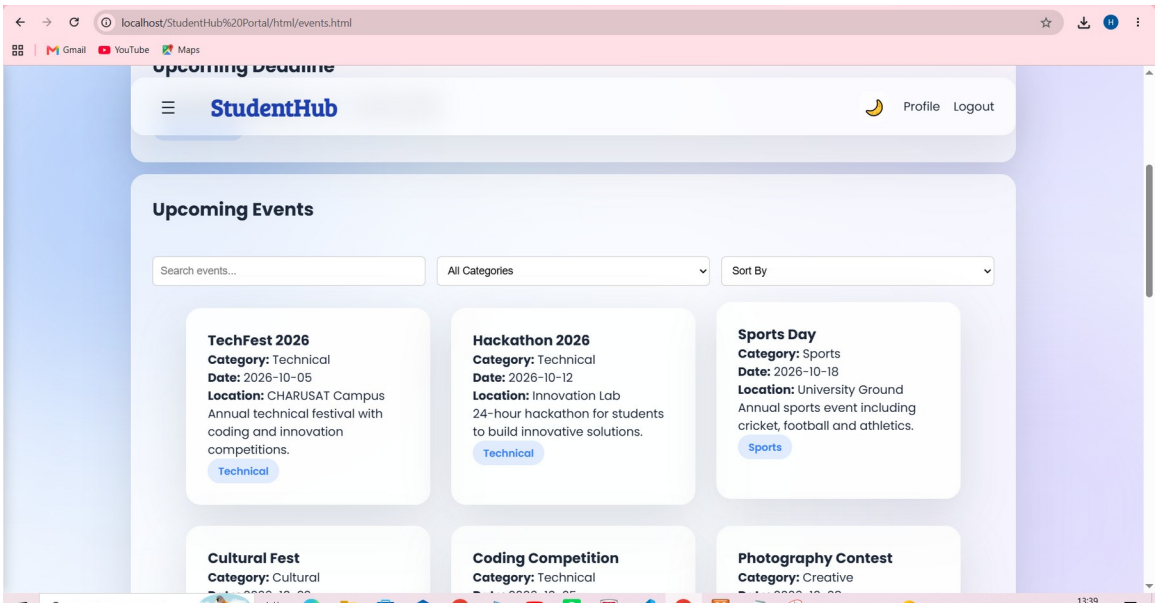
C. Evidence of Work (Screenshots)

Figure 1.1 JSON file of events

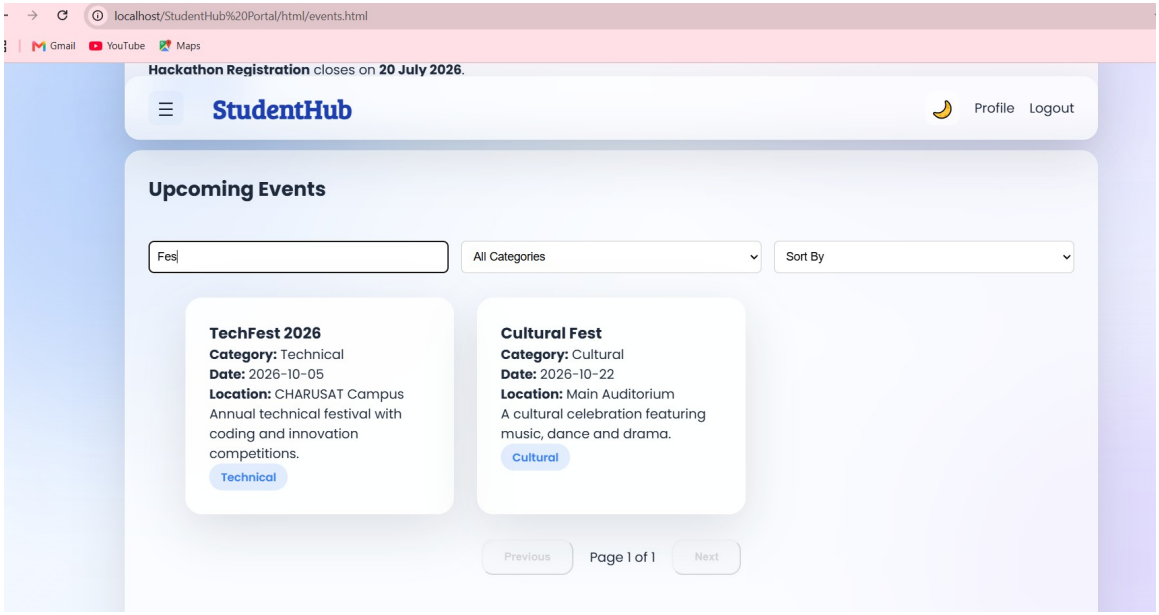


Description : Json file that has data of events

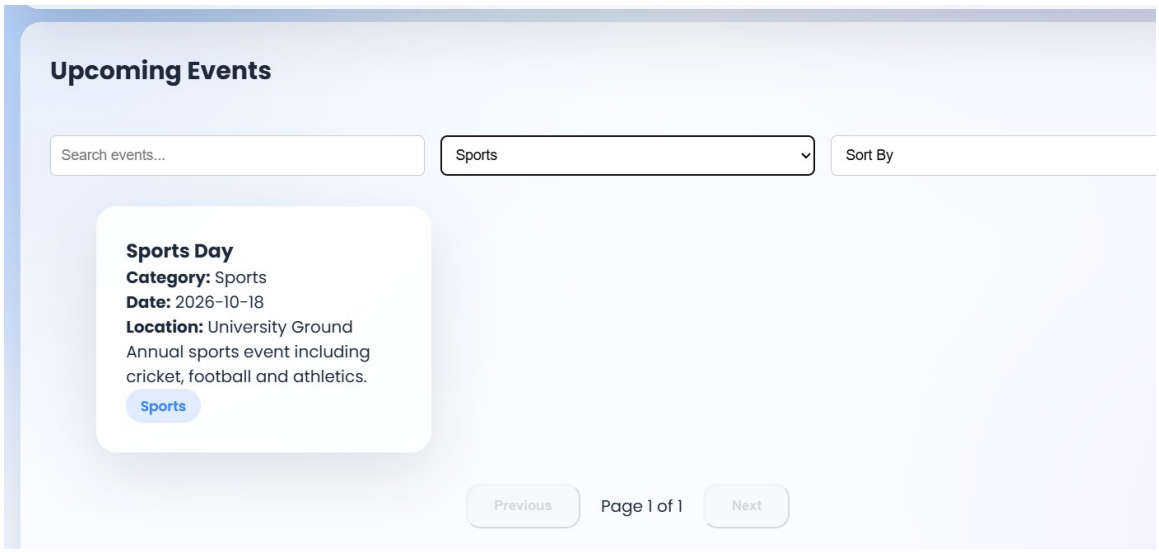
Figure 1.2 Event page



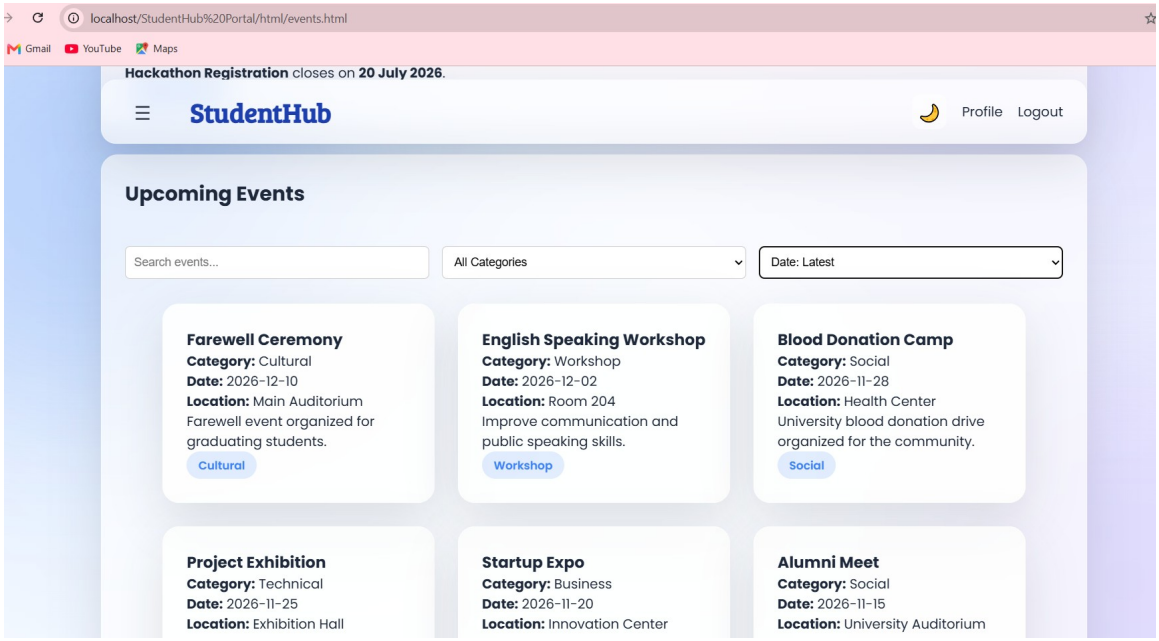
Description: Events displayed from json file



Description: Search Functionality



Description: Category wise filtering



Description: Sorting

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

Initially, I faced difficulty in fetching the JSON files correctly because of incorrect file path , but that was done correctly afterwords.

F. Learning Outcomes:

- Learned how to fetch and use external JSON data with the Fetch API.
- Learned how to parse JSON and dynamically display data using JavaScript.
- Understood the use of map(), filter(),sort() and array slicing for data manipulation.
- Learned how to implement search, filtering in a web page.
- Understood how to handle loading and error states in a web application.
- Learned how to organize JavaScript code into reusable functions.
- Gained practical experience in creating dynamic and reusable frontend views.

Faculty **Remarks:**

Practical - 7

Practical Title: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Problem Definition: Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.

A. Theory

1. Is the form submitted using POST?

Answer: Yes, the form data is submitted using the POST method. PHP receives the submitted values through $_POST, which allows the data to be processed on the server without putting the form values in the URL.

2. Are inputs validated and sanitized server side?

Answer: Yes, the submitted data is validated and sanitized in PHP before storing it. Required fields are checked, and values such as email, mobile number, and password are checked for the correct format. Unwanted spaces or unsafe input can also be removed or handled properly.

3. Is file writing handled safely?

Answer: Yes, the validated data is stored in a CSV or JSON file using PHP file-handling functions. The file is opened or updated carefully so that existing records are not accidentally lost. Data is converted into the correct format before writing.

4. Are success and error responses displayed clearly?

Answer: Yes, after processing the form, PHP displays a success message when the data is stored correctly. If validation fails or there is a problem while saving the data, an appropriate error message is displayed so the user knows what needs to be corrected.

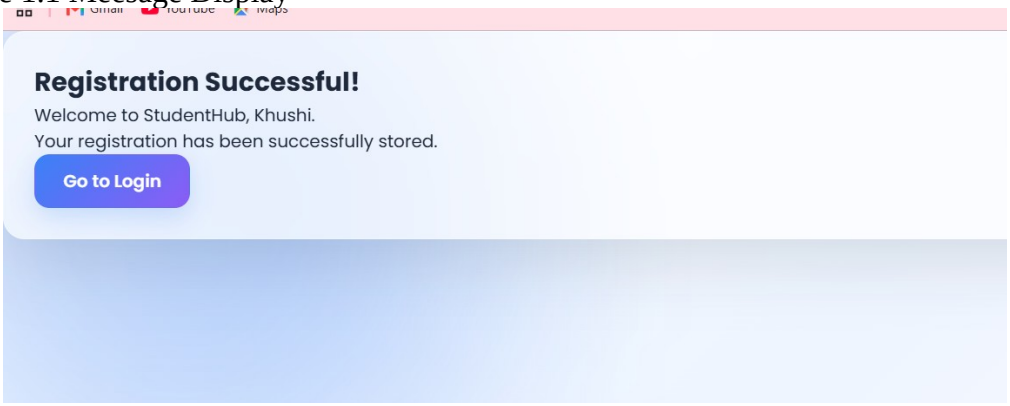
B. Implementation/ Output/Result

1. Created an HTML registration form to collect user details.
2. Used the POST method to submit the form data securely to the PHP processor.
3. Used PHP's $_POST to receive the submitted form values.
4. Added server-side validation to check required fields and validate inputs such as email, mobile number, and password.
5. Used PHP file-handling functions to store the records safely in the data file.
6. Displayed a success message when the record was stored successfully.
7. Displayed appropriate error messages when validation or file processing failed.

8. Tested the form with both valid and invalid inputs to verify the complete process.

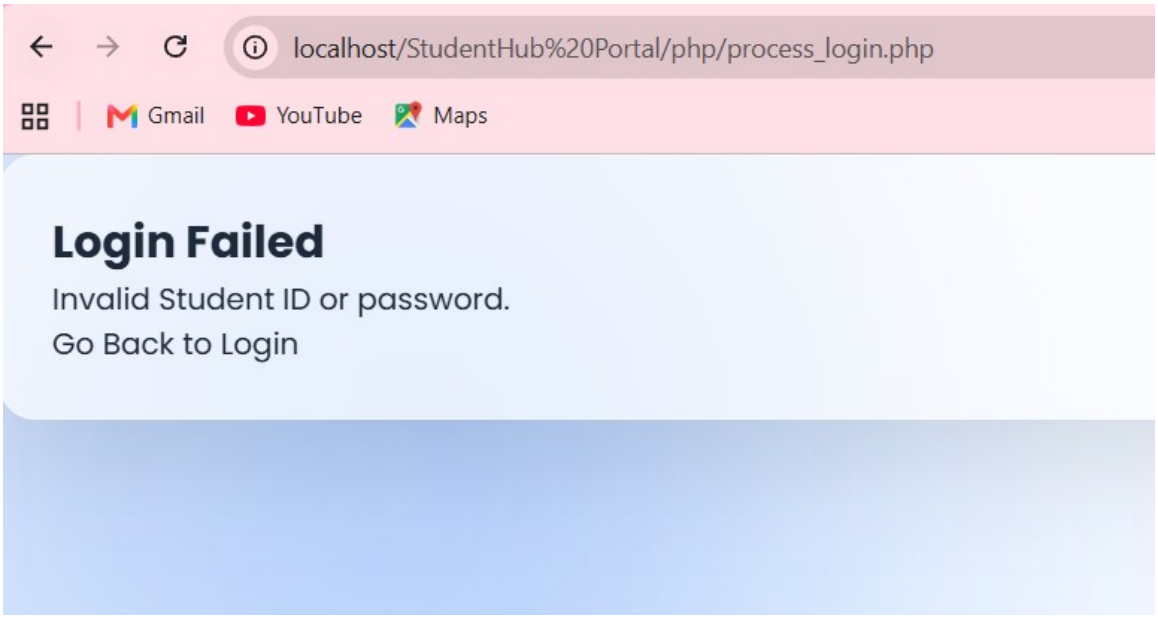
C. Evidence of Work (Screenshots)

Figure 1.1 Meesage Display



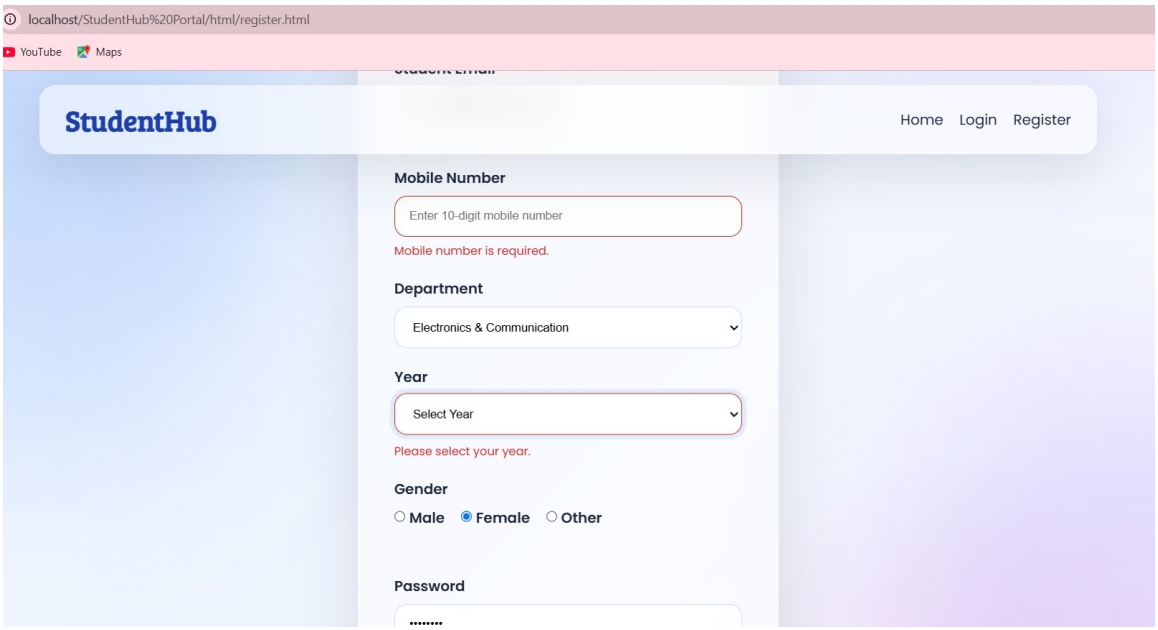
Description: Successful message is displayed when registration is successful.

Figure 1.2 Failed login display page



Description: Displaying message when login fails

Figure 1.3 Error message



Description: Message is displayed when fields are empty or incorrect.

Figure 1.1 JSON file



Description: JSON file that stores data from registration form

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

- Handling incorrect or empty inputs without causing errors was a challenge.
- Making sure that old records were not overwritten while adding new records required careful file handling.

F. Learning Outcomes:

- Learned how to process HTML forms using PHP and the POST method.
- Understood the importance of server-side validation and input sanitization.
- Learned how to receive and process form data using $_POST.
- Learned how to store form records in JSON files using PHP.

Faculty Remarks:

Practical - 8

Practical Title: MySQL Schema Design, ER Model, PDO Connectivity, and Prepared Statements

Problem Definition: Design the database for StudentHub with students, events and registrations. Create PHP database connection using MYSQL statements.

A. Theory

1. Is the schema normalized properly?

Answer: Yes, the StudentHub database is normalized by separating the data into three tables: students, events, and registrations. The students table stores student information, the events table stores event details. This reduces data redundancy, avoids unnecessary duplication, and helps maintain data consistency. .

2. Are primary keys and foreign keys correctly defined?

Answer:Yes, primary keys and foreign keys are used to maintain data integrity. The student_id in the students table and event_id in the events table uniquely identify each record. The registration_id in the registrations table uniquely identifies each registration. Foreign keys in the registrations table reference student_id and event_id, ensuring that registrations are associated with existing students and events.

3. Is the database connection handled securely?

Answer: Write your answer in your own words. Do not copy and paste content directly from AI tools. At the end of your answer, provide the website(s) or reference link(s) you used for research.

B. Implementation

Database connection using PHP PDO (db.php)

```
<?php

$host = "localhost";

$dbname = "studenthub";

$username = "root";

$password = "";

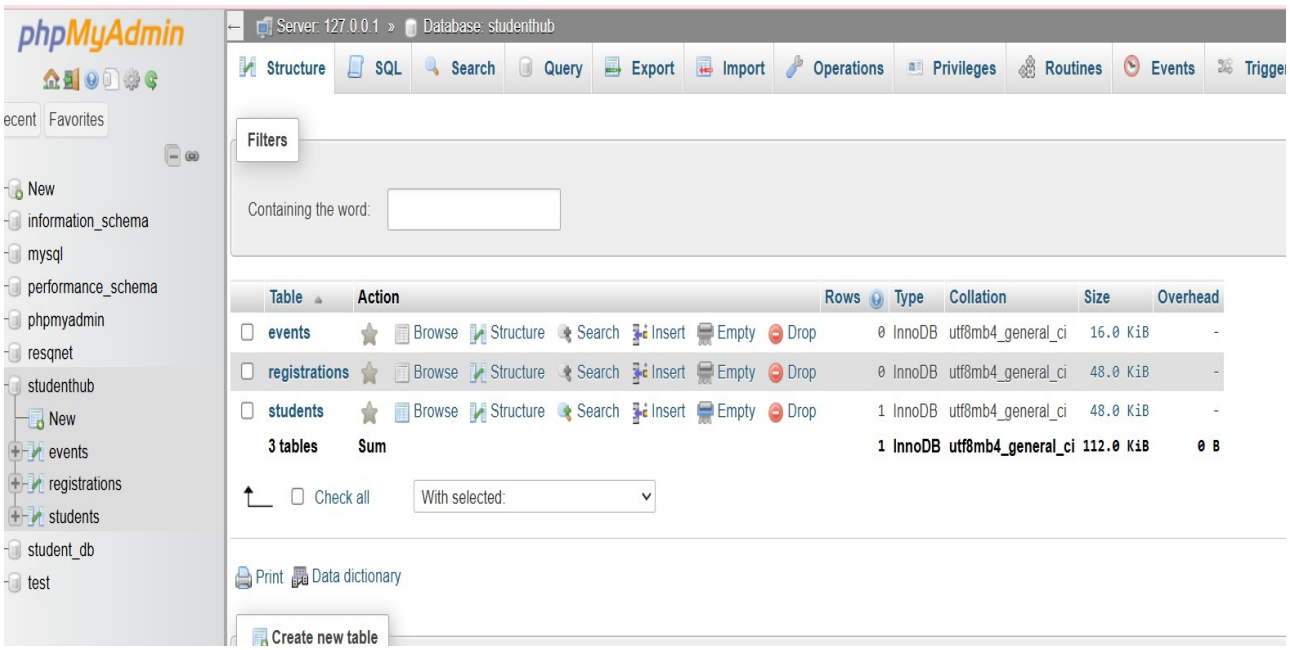
try {

   $pdo = new PDO(
```

```
        "mysql:host=dbname;charset=utf8mb4",
        $username,
        $password
    );
    $pdo->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
    );
    echo "Database connected successfully!";
} catch (PDOException $e) {
    die("Database connection failed.");
}
?>
```

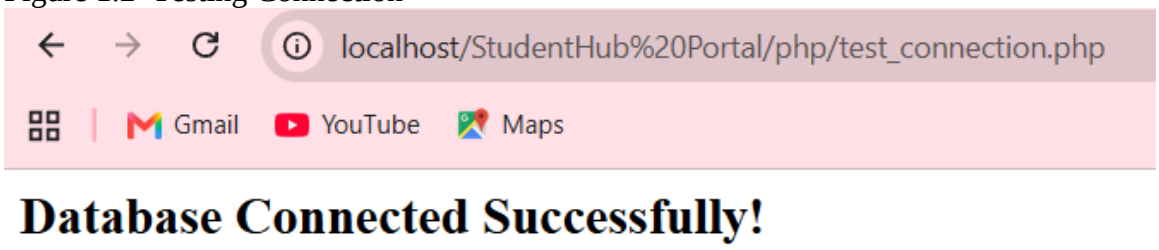
C. Evidence of Work (Screenshots)

Figure 1.1 Database in phpMyAdmin



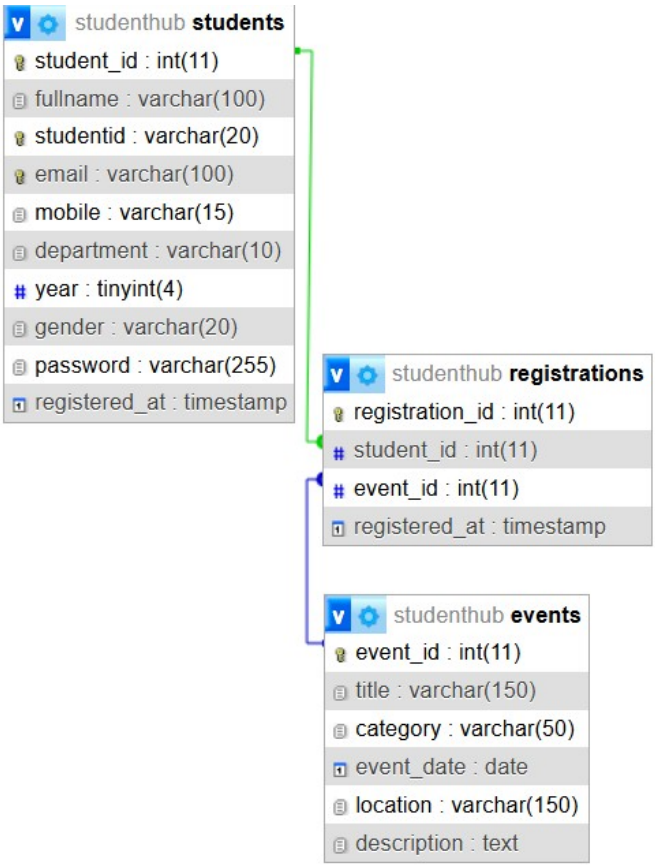
Description: Students, events and registration table

Figure 1.2 Testing Connection



Description: Successful message display

Figure 1.3 ER diagram



Description: ER diagram showing tables, columns and relations

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

Defining primary keys and foreign keys correctly to maintain data integrity.

F. Learning Outcomes:

- Learned how to design a MySQL database schema and create tables with appropriate constraints.
- Understood database normalization and the importance of primary keys and foreign keys.
- Learned to connect PHP with MySQL using PDO.

Faculty Remarks:

Practical - 9

Practical Title: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition: Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

A. Theory

1. Is password_hash() used before storing passwords?

Answer: Yes, password_hash() is used to generate a secure password hash before storing the password in the database. This prevents passwords from being stored as plain text.

2. Is duplicate email/username checked?

Answer: Yes, the database is checked for existing email addresses before inserting a new user. This helps prevent duplicate accounts. Unique constraints in the database provide additional protection against duplicates.

3.Are frontend and backend validations consistent?

Answer: Yes, frontend validation provides immediate feedback to users, while backend validation checks the submitted data again before processing it. Both validations should follow the same rules to ensure data consistency and sec-urity.

B. Implementation

Storing passwords

```
$passwordHash =  password_hash( $password, PASSWORD_DEFAULT);
```

Checking for duplicates

```
$checkQuery = "

    SELECT student_id

    FROM students
```

```
    WHERE studentid = :studentid

       OR email = :email

    LIMIT 1

";

$checkStmt = $pdo->prepare($checkQuery);

$checkStmt->execute([

    ":studentid" => $studentid,

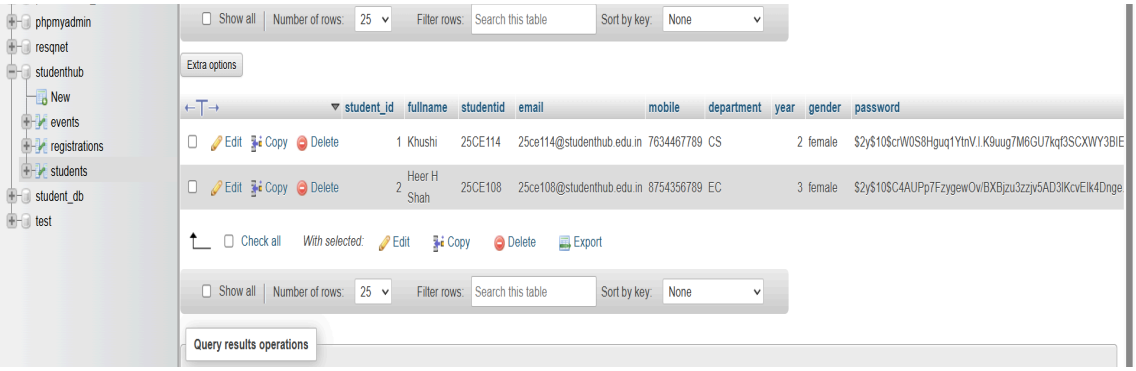
    ":email" => $email

]);

$existingStudent = $checkStmt->fetch();
```

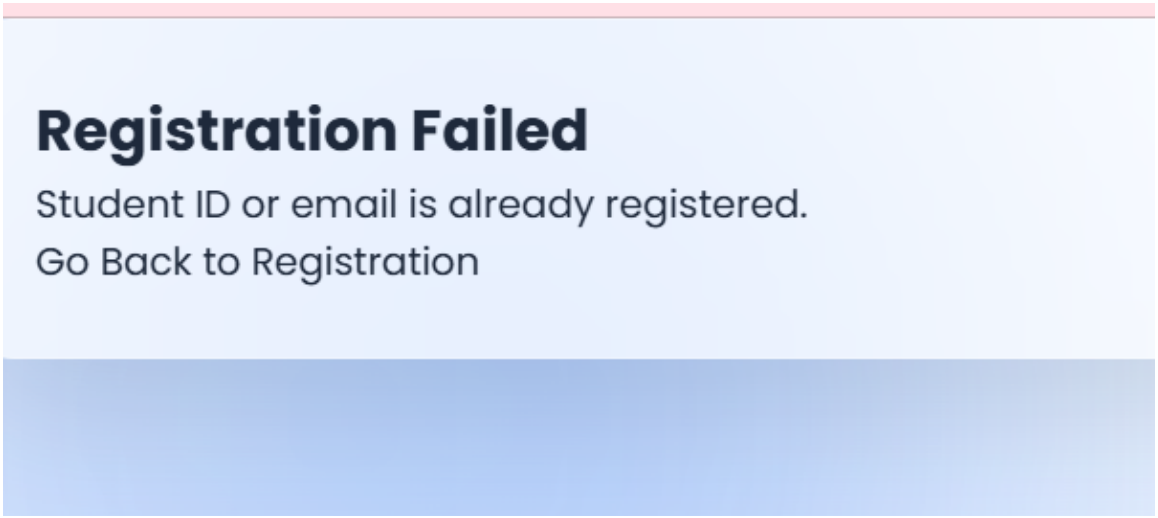
C. Evidence of Work (Screenshots)

Figure 1.1 Database in phpmyadminn



Description: Passwords are stored in hash

Figure 1.2 Message showing



Description: When we try registering duplicate email, this message shows and registration fails

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

Checking for duplicate email addresses and usernames before inserting new users into the database.

F. Learning Outcomes:

- Learned how to implement secure user registration using PHP and MySQLi.
- Learned to check duplicate email addresses and usernames before inserting records.
- Understood how to use password_hash() to securely hash passwords before storing them in the database.

Faculty Remarks:

Practical - 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

A. Theory

1. Are sessions started and destroyed securely?

Answer: Yes, PHP sessions are started using session_start(). The session ID is regenerated after successful login using session_regenerate_id(true). During logout, session data is cleared, the session cookie is removed, and the session is destroyed using session_destroy().

2. Is password_verify() used correctly?

Answer: Yes, password_verify() compares the password entered by the user with the hashed password stored in the database. This allows secure password verification without storing plain-text passwords.

3. Is unauthorized dashboard access blocked?

Answer: Yes, dashboard pages check whether the user has an active authenticated session. If a user is not logged in, they are redirected to the login page. Role-specific pages also verify the user's role before granting access.

4.Is session timeout implemented?

Answer: Yes, a session timeout can be implemented by tracking the user's last activity timestamp. If the user remains inactive beyond the configured time limit, the session is cleared and the user must log in again.

5. Are admin and student users redirected to appropriate dashboards?

Answer: Yes, role-based redirection sends administrators to the admin dashboard and students to the student dashboard after successful authentication.

B. Implementation

Verify password and regenerate session ID

```
session_start();

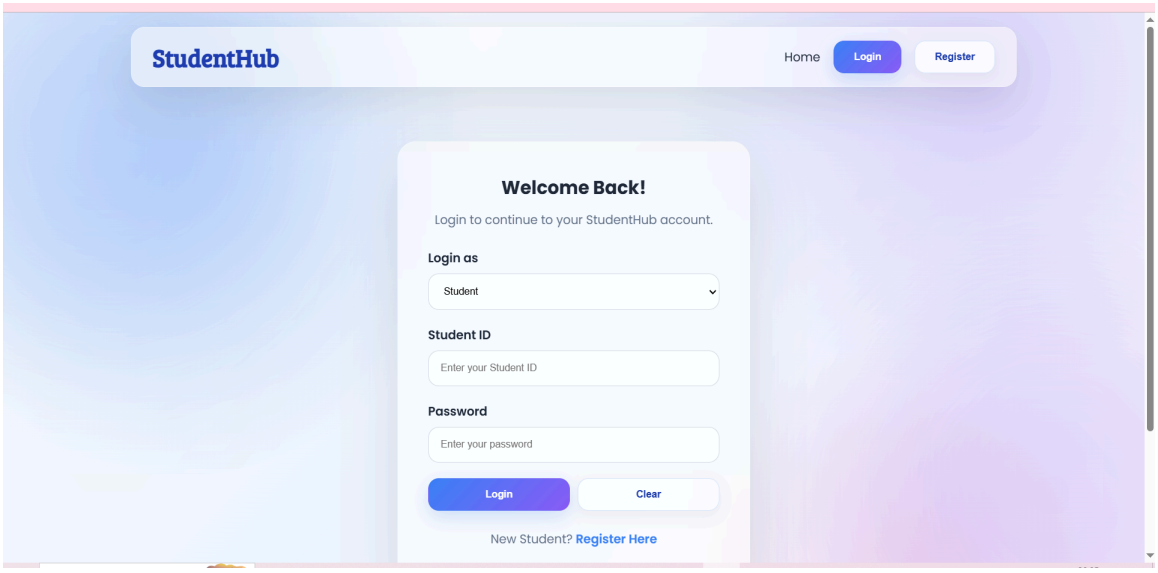
if (password_verify($password, $user['password'])) {
    session_regenerate_id(true);

    $_SESSION['user_id'] = $user['id'];
    $_SESSION['role'] = $user['role'];

    if ($user['role'] === 'admin') {
        header("Location: admin_dashboard.php");
    } else {
        header("Location: student_dashboard.php");
    }
    exit;
} else {
    echo "Invalid email or password.";
}
```

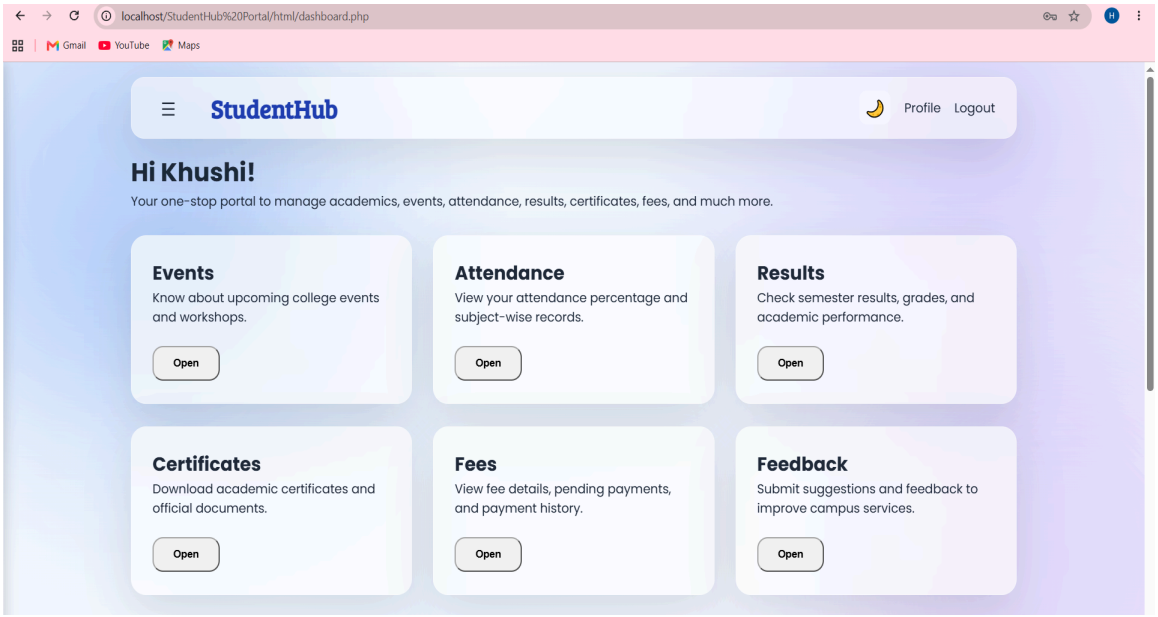
C. Evidence of Work (Screenshots)

Figure 1.1 Login page

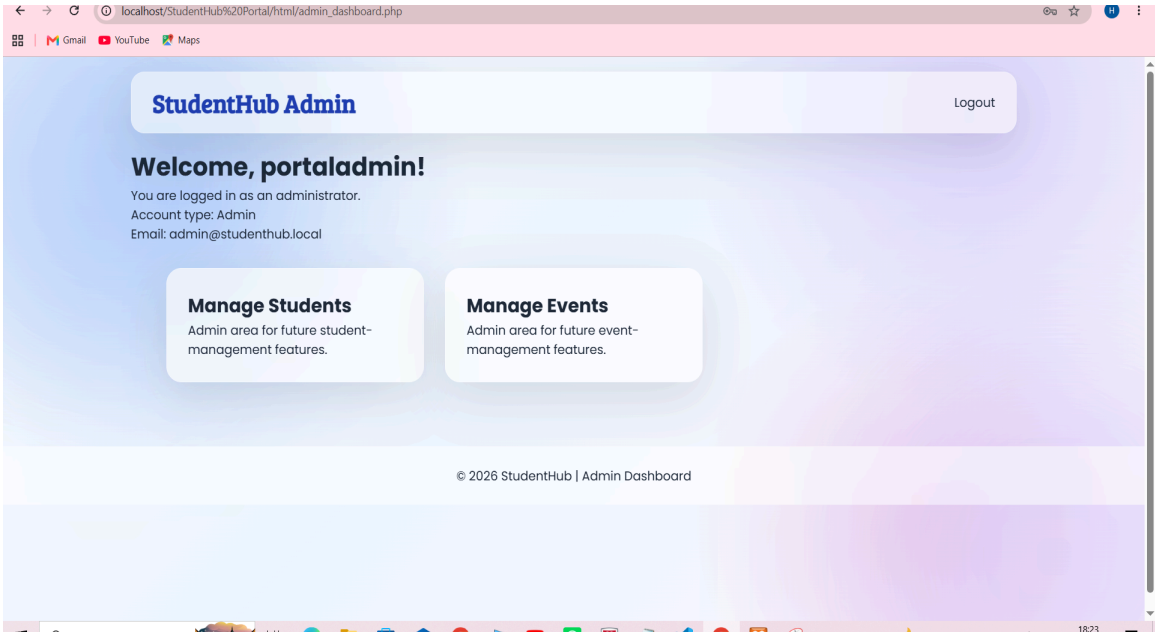


Description: Login asks for role and if successful, navigates to related dashboard

Figure 1.2 Dashboard

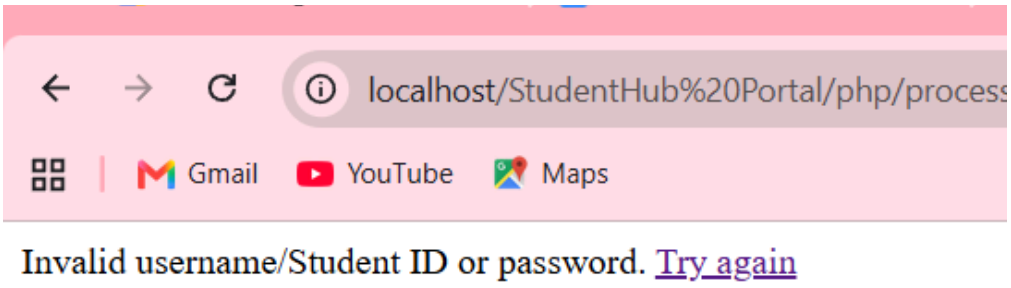


Description: Student's dashboard

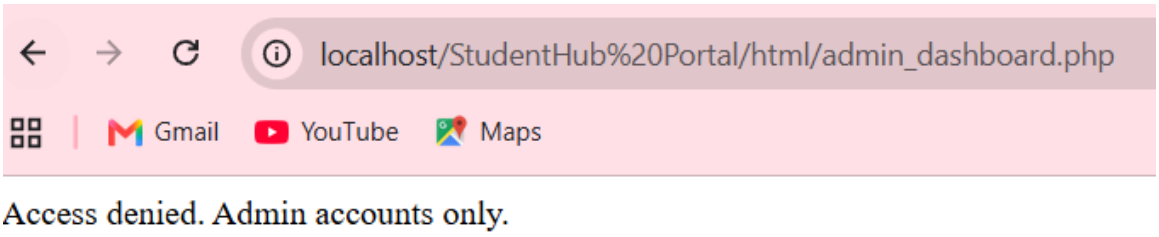


Description: Admin's Dashboard

Figure 1.3 Message



Description: Shows message when typed wrong id or password



Description: Shows message when trying to access admin dashboard while logged in as student

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

- Configuring role-based redirection for administrators and students.

F. Learning Outcomes:

- Learned how to implement login and logout functionality using PHP sessions.
- Understood password verification and secure authentication practices.

Faculty Remarks:

Practical - 11

Practical Title: Student Management CRUD with Search and Filter Problem definition: Create a student management module to add, view, edit, delete, search and filter student records. All database operations must use MYSQLi statements and clear success/failure messages.

A. Theory:

1. Are SELECT, INSERT, UPDATE, and DELETE operations working?

   Answer: Yes, all CRUD operations are working correctly. SELECT displays student records, INSERT adds new students, UPDATE modifies student details, and DELETE removes student records.

2. Is search by name/email/course implemented?

   Answer: Yes, search by student name, email, and course is implemented. The course filter also displays students from the selected course.

3. Are delete operations confirmed before execution?

   Answer: Yes, a confirmation message appears before deleting a student record. The record is deleted only after clicking OK.

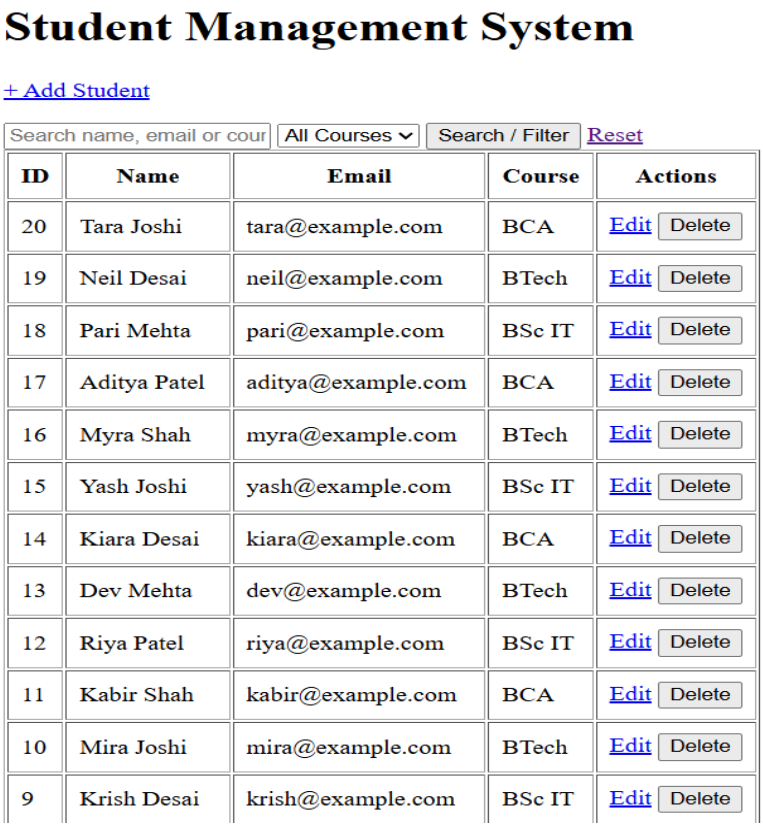
A. Implementation

PHP Functions and MySQLi Methods Used

Function / Method	Purpose
new mysqli()	Connects PHP to MySQL.
prepare()	Prepares an SQL statement.
bind_param()	Binds values to query parameters.
execute()	Executes the SQL statement.
get_result()	Retrieves query results.
fetch_assoc()	Fetches a row as an associative array.
trim()	Removes extra whitespace from input.
htmlspecialchars()	Helps safely display user-provided text.
confirm()	Asks for confirmation before deletion.
isset()	Checks whether a variable or form value exists.

Code	Purpose
$conn = new mysqli(...)	Connects PHP to MySQL
INSERT INTO	Adds a new student
SELECT	Retrieves student records
UPDATE	Modifies student information
DELETE FROM	Removes a student
$conn->prepare()	Prepares an SQL statement safely
bind_param()	Binds values to the query parameters
LIKE	Searches for matching text
LIMIT and OFFSET	Display records page by page
confirm()	Asks the user before deletion
htmlspecialchars()	Helps prevent HTML/script injection when displaying data
filter_var()	Validates the email format

B. Evidence of Work (Screenshots)



Student Management System

+ Add Student

Search name, email or cour | All Courses | Search / Filter | Reset

ID	Name	Email	Course	Actions
20	Tara Joshi	tara@example.com	BCA	Edit Delete
19	Neil Desai	neil@example.com	BTech	Edit Delete
18	Pari Mehta	pari@example.com	BSc IT	Edit Delete
17	Aditya Patel	aditya@example.com	BCA	Edit Delete
16	Myra Shah	myra@example.com	BTech	Edit Delete
15	Yash Joshi	yash@example.com	BSc IT	Edit Delete
14	Kiara Desai	kiara@example.com	BCA	Edit Delete
13	Dev Mehta	dev@example.com	BTech	Edit Delete
12	Riya Patel	riya@example.com	BSc IT	Edit Delete
11	Kabir Shah	kabir@example.com	BCA	Edit Delete
10	Mira Joshi	mira@example.com	BTech	Edit Delete
9	Krish Desai	krish@example.com	BSc IT	Edit Delete

Student Management System

+ Add Student

Sara | All Courses | Search / Filter | Reset

ID	Name	Email	Course	Actions
8	Sara Mehta	sara@example.com	BCA	Edit Delete

C. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal/tree/ed9b309f7c40e08097f20a8c86c3b8f7277bc24a/practical11

D. Challenges Faced:

1. Performing INSERT, SELECT, UPDATE, and DELETE operations correctly.
2. Implementing search by name, email, and course.
3. Displaying proper success and error messages.

Learning Outcomes:

1. Understood how to perform CRUD operations on student records.
2. Learned to search students by name, email, and course.
3. Understood how to filter records based on the selected course.

Faculty Remarks:

Practical - 12

Practical Title: Event Management CRUD with Poster Upload and File Validation

Problem definition: Develop event management functionality for admin users. Admin can add, view, update, delete, and upload event posters with file type and file size validation.

A.Theory:

1. Are add, update, and delete functionalities implemented correctly?

   Answer: Yes, the add, update, and delete functionalities are implemented to manage event records in the database

2. Is the UI linked with the database correctly?

   Answer: Yes, the user interface is connected to the MySQL database using PHP, allowing event records to be displayed and managed.

3. Are poster uploads validated for type and size?

   Answer: Poster upload validation should check the file type and size before uploading. If these checks are implemented in the code, invalid files are rejected.

4. Are success and failure messages displayed?

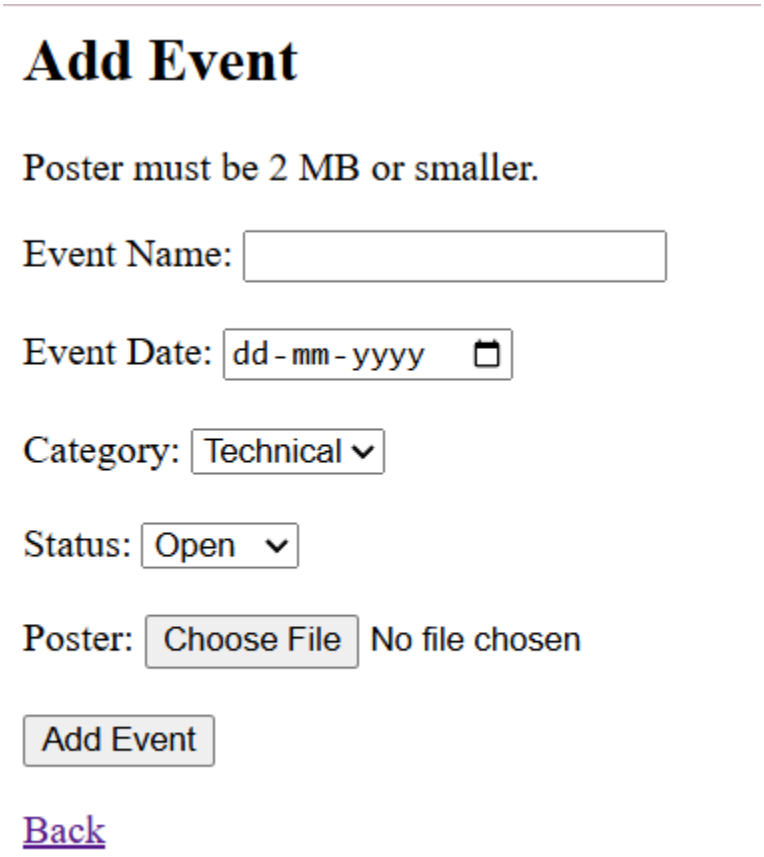
   Answer: Yes, success and failure messages can be displayed to inform users whether an operation was completed successfully or an error occurred.

A. Implementation

Important PHP functions and features

Function / Feature	Purpose
session_start()	Starts or resumes a session.
isset()	Checks whether a variable or value exists.
mysqli()	Connects PHP to the MySQL database.
prepare()	Prepares an SQL statement.
bind_param()	Binds values to a prepared SQL statement.
execute()	Executes the SQL statement.
move_uploaded_file()	Saves an uploaded poster to the upload folder.

B. Evidence of Work (Screenshots)



C. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal/tree/ed9b309f7c40e08097f20a8c86c3b8f7277bc24a/Practical12

D. Challenges Faced:

- Implementing add, view, update, and delete operations for event records.
- Validating poster images based on file type and file size.

Learning Outcomes:

- Learned how to perform CRUD operations on event records.

Faculty Remarks: **(Keep this blank for faculty)**

Practical - 13

Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Theory:

1. Do PHP endpoints return valid JSON responses?

   Answer: Yes, PHP endpoints return valid JSON responses using json_encode(). The frontend receives and processes the JSON data.

2. Are GET and POST requests handled properly?

   Answer: Yes, GET requests are used to display student records, while POST requests are used to add, update, and delete student records.

3. Are loading, success, and error states shown on the frontend?

   Answer: Yes, the frontend displays loading messages, success messages after operations, and error messages when something goes wrong.

4. Is input validation performed before database operations?

   Answer: Yes, the program checks that all required fields are filled and that the email address is valid before performing database operations.

A. Implementation/ Output/Result (Keep only one option which is suitable to practical)

Tags Used and Their Purpose

Tag	Purpose
`<html>`	Defines the HTML document.
`<head>`	Contains page information and styles.
`<title>`	Sets the webpage title.
`<style>`	Adds CSS styling to the webpage.
`<body>`	Contains the visible webpage content.
`<h1>`	Displays the main heading.

`<form>`	Creates the student input form.
`<input>`	Accepts student details.
`<button>`	Performs add, update, clear, and delete actions.
`<p>`	Displays loading, success, and error messages.
`<table>`	Displays student records.
`<thead>`	Defines the table headings.
`<tbody>`	Contains student records added dynamically.
`<tr>`	Defines a table row.
`<th>`	Defines a table heading cell.
`<td>`	Defines a table data cell.
`<script>`	Contains JavaScript code for Fetch API and CRUD operations.

B. Evidence of Work (Screenshots)

Student Management — Fetch API

Students loaded successfully.

Student name | Email | BCA | Save Student | Clear

Student List

ID	Name	Email	Course	Actions
3	Rohan Mehta	rohan@example.com	BTech	Edit Delete
2	Diya Patel	diya@example.com	BSc IT	Edit Delete
1	Aarav Shah	aarav@example.com	BCA	Edit Delete

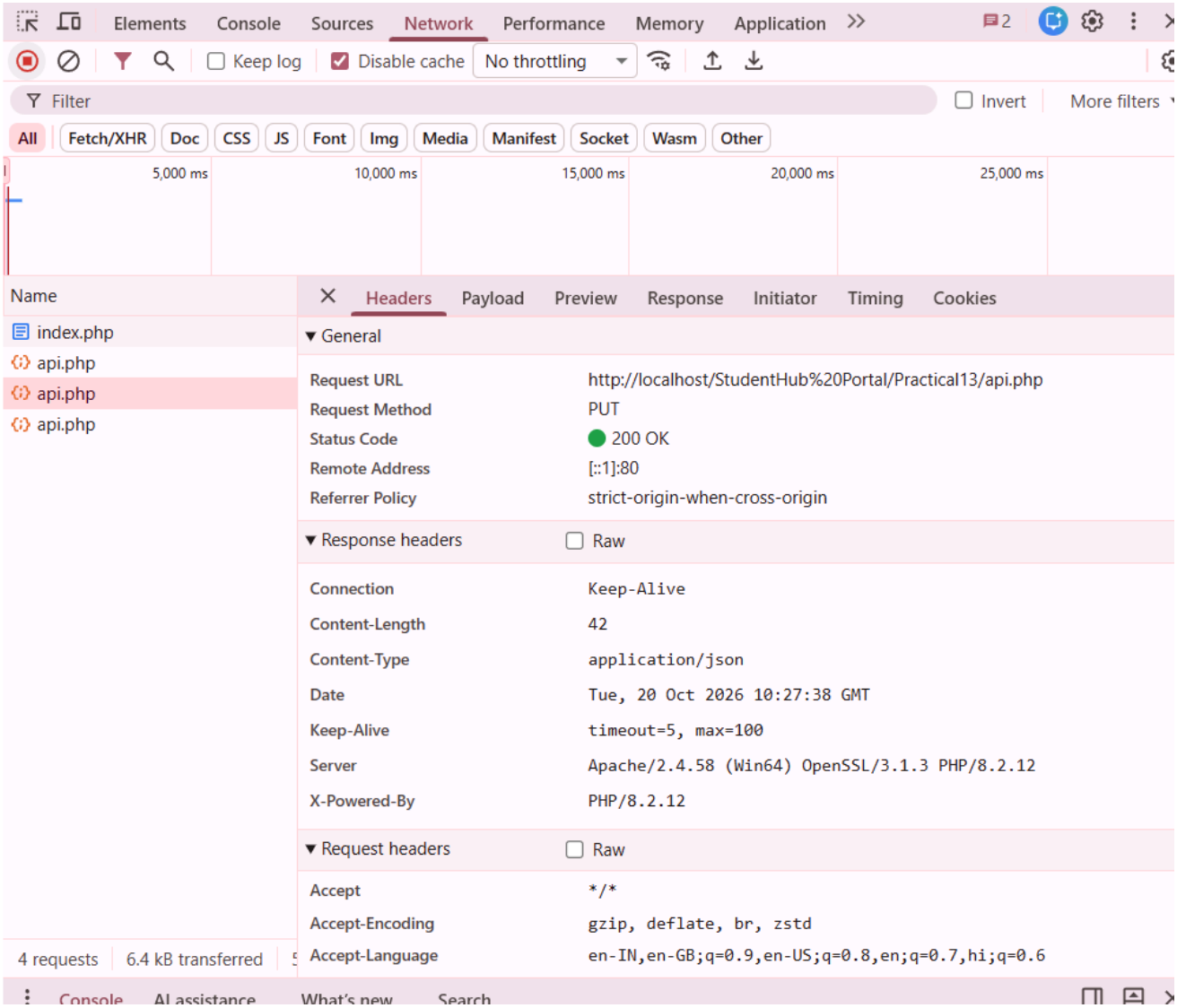
Student Management — Fetch API

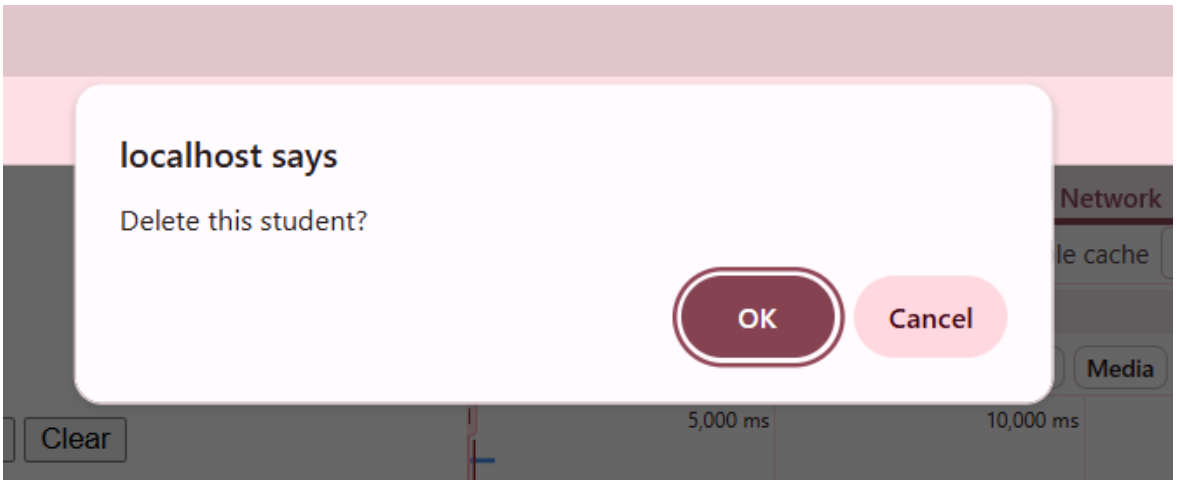
Students loaded successfully.

Student name | Email | BCA | Save Student | Clear

Student List

ID	Name	Email	Course	Actions
4	Heer	heershaah@gmail.com	BTech	Edit Delete
3	Rohan Mehta	rohan@example.com	BTech	Edit Delete
2	Diya Patel	diya@example.com	BSc IT	Edit Delete
1	Aarav Shah	aarav@example.com	BCA	Edit Delete


Elements
Console
Sources
Network
Performance
Memory
Application
Keep log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
5,000 ms
10,000 ms
15,000 ms
20,000 ms
25,000 ms
Name
index.php
api.php
api.php
api.php
Headers
Payload
Preview
Response
Initiator
Timing
Cookies
General
Request URL
http://localhost/StudentHub%20Portal/Practical13/api.php
Request Method
PUT
Status Code
200 OK
Remote Address
[::1]:80
Referrer Policy
strict-origin-when-cross-origin
Response headers
Raw
Connection
Keep-Alive
Content-Length
42
Content-Type
application/json
Date
Tue, 20 Oct 2026 10:27:38 GMT
Keep-Alive
timeout=5, max=100
Server
Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
X-Powered-By
PHP/8.2.12
Request headers
Raw
Accept
/
Accept-Encoding
gzip, deflate, br, zstd
Accept-Language
en-IN,en-GB;q=0.9,en-US;q=0.8,en;q=0.7,hi;q=0.6
4 requests
6.4 kB transferred
Console
AI assistance
What's new
Search


localhost says
Delete this student?
OK
Cancel
Network
Media
Clear
5,000 ms
10,000 ms

C. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

D. Challenges Faced:

- Connecting PHP API endpoints with the MySQL database.
- Sending and receiving data using JSON and Fetch API.
- Performing CRUD operations without reloading the webpage.

Learning Outcomes:

- Learned how to create PHP JSON API endpoints.
- Learned how to use Fetch API for asynchronous communication.

Faculty Remarks:

Practical - 14

Practical Title: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Problem definition: Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role.

Theory:

1. Are dashboard values loaded dynamically from the database?

   Yes. PHP executes SQL COUNT(*) queries to display the current number of users, students, and events.

2. Is access restricted to admin users?

   Yes. PHP sessions check whether the user is logged in and has the admin role before displaying the dashboard.

3. Are role-based menus working?

   Yes. Admin users see administrative links, while student users see only student-related links.

4. Are important admin actions recorded in audit_logs?

   Yes. Opening the dashboard inserts an action, the admin's user ID, and the date and time into the audit log.

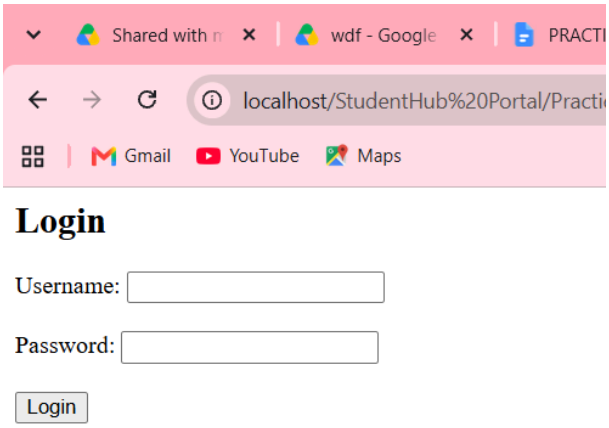
A. Implementation

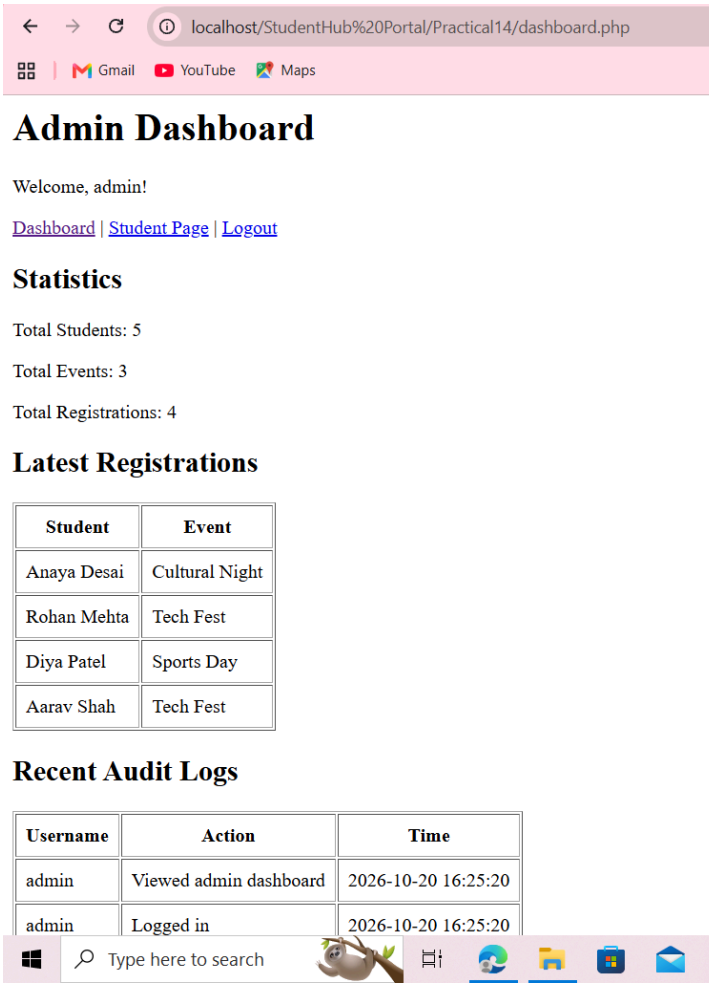
PHP functions and methods used

Function / method	Purpose
`session_start()`	User session start kare chhe.
`new mysqli()`	MySQL database sathe connection banave chhe.
`connect_error`	Database connection error check kare chhe.
`prepare()`	SQL query safely prepare kare chhe.
`bind_param()`	Query ma values bind kare chhe.
`execute()`	SQL query execute kare chhe.
`password_verify()`	Enter karelo password stored hash sathe check kare chhe.
`password_hash()`	Password ne secure hash ma convert kare chhe.

`header()`	Redirect karva mate.
`session_destroy()`	Session destroy kari logout kare chhe.
`htmlspecialchars()`	HTML output ma unsafe characters escape kare chhe.
`COUNT(*)`	Total users, students ane events count kare chhe.
`fetch_assoc()`	Database result ne associative array ma lave chhe.

B. Evidence of Work (Screenshots)





C. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

D. Challenges Faced:

- Connecting PHP with the MySQL database.
- Fetching counts using SQL queries.
- Restricting the dashboard to admin users.
- Displaying different menus according to user roles.
- Recording admin activities in the audit log.

E. Learning Outcomes:

I learned how to create an admin dashboard using PHP and MySQL.

I learned to fetch dynamic statistics using SQL queries, restrict access using role-based access control, display menus according to user roles, and record important activities in an audit log.

Faculty Remarks:

Practical - 15

Practical Title: Final Integration, Testing, Documentation, GitHub Submission, and Local Deployment

Problem definition: Integrate all modules into a single deployable StudentHub portal. Verify navigation, responsiveness, authentication, database operations, security checks, AJAX functionality, and documentation. Deploy locally using XAMPP/Apache virtual host and submit GitHub repository.

A. Theory:

● Are all pages properly linked and navigable?

Yes, all required pages are linked through the navigation menu, and each page opens correctly without broken links.

● Are sessions, roles, validations, and database operations working end-to-end?

Yes, after testing, registration, login, logout, session management, role-based access, form validation, and database operations work correctly.

● Are security and accessibility checklists completed?

The application is checked for password hashing, prepared statements, session protection, input validation, image alt text, form labels, keyboard navigation, and readable contrast.

● Is project documentation sufficient for installation and testing?

Yes, the README explains the technologies, installation steps, database setup, features, and testing process. The project also includes an SQL dump and test report.

● Is the project repository clean and properly committed?

Yes, the required project files are committed and pushed to GitHub, and temporary files and sensitive credentials are excluded.

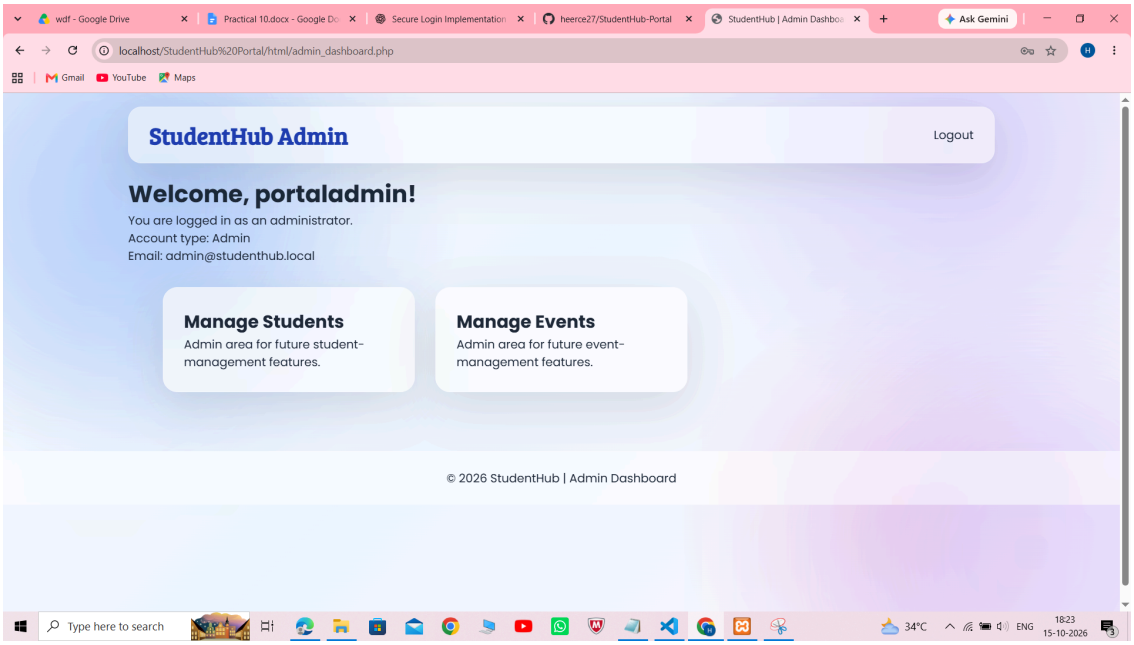
B. Implementation

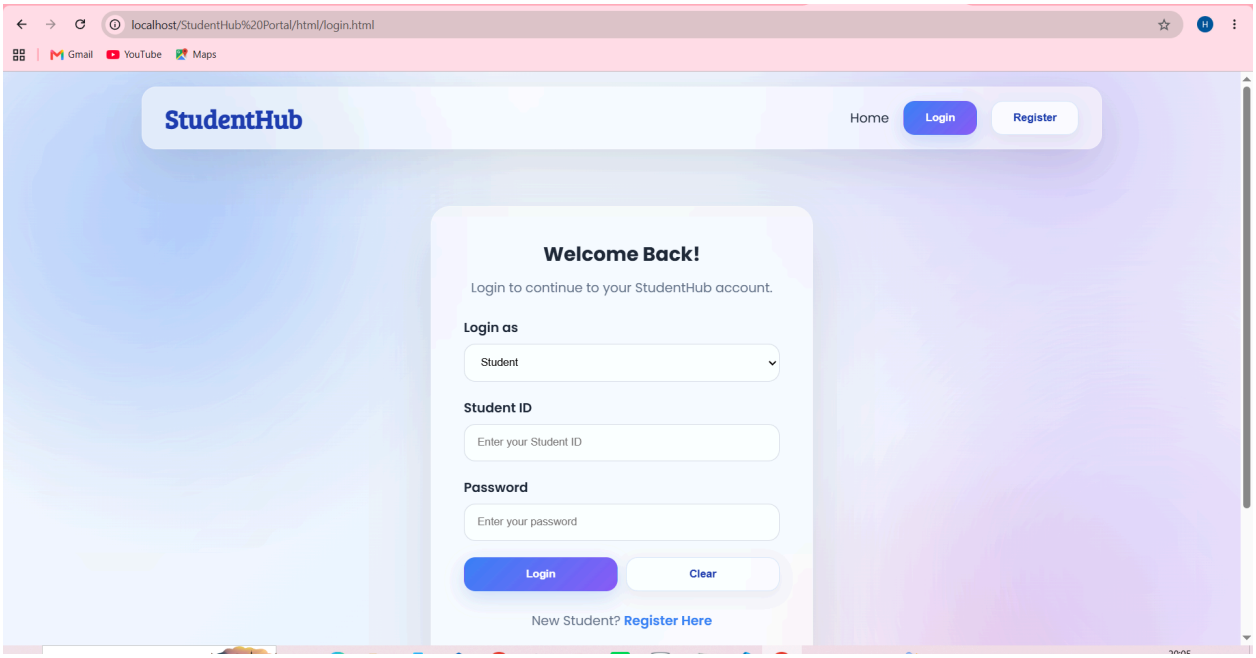
PHP syntax and functions used:

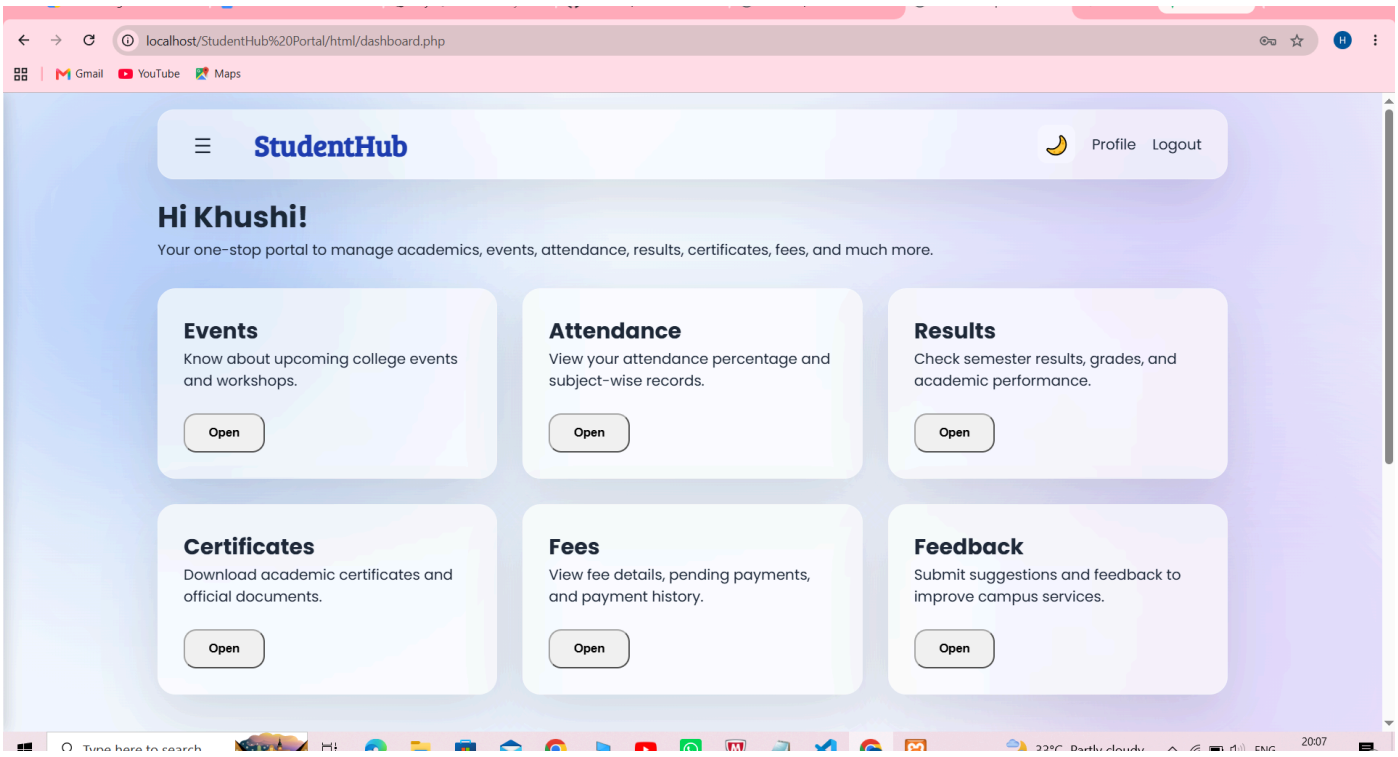
PHP element	Purpose

`require_once`	Includes the database connection file only once.
`new mysqli()`	Connects PHP to the MySQL database.
`if` and `else`	Check conditions and display appropriate output.
`while`	Repeats code to display each student record.
`fetch_assoc()`	Retrieves a database row as an associative array.
`htmlspecialchars()`	Helps prevent HTML injection when displaying database values.
`close()`	Closes the database connection.
`$_SERVER`	Provides server and request information when required.

C. Evidence of Work




localhost/StudentHub%20Portal/html/login.html
Gmail
YouTube
Maps
StudentHub
Home
Login
Register
Welcome Back!
Login to continue to your StudentHub account.
Login as
Student
Student ID
Enter your Student ID
Password
Enter your password
Login
Clear
New Student? Register Here


localhost/StudentHub%20Portal/html/dashboard.php
Gmail
YouTube
Maps
StudentHub
Profile
Logout
Hi Khushi!
Your one-stop portal to manage academics, events, attendance, results, certificates, fees, and much more.
Events
Know about upcoming college events and workshops.
Open
Attendance
View your attendance percentage and subject-wise records.
Open
Results
Check semester results, grades, and academic performance.
Open
Certificates
Download academic certificates and official documents.
Open
Fees
View fee details, pending payments, and payment history.
Open
Feedback
Submit suggestions and feedback to improve campus services.
Open

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

- Login issues were fixed by verifying sessions and role checks.

Learning Outcomes:

After completing this practical, I learned how to integrate different web development modules into a single application.

Faculty Remarks: